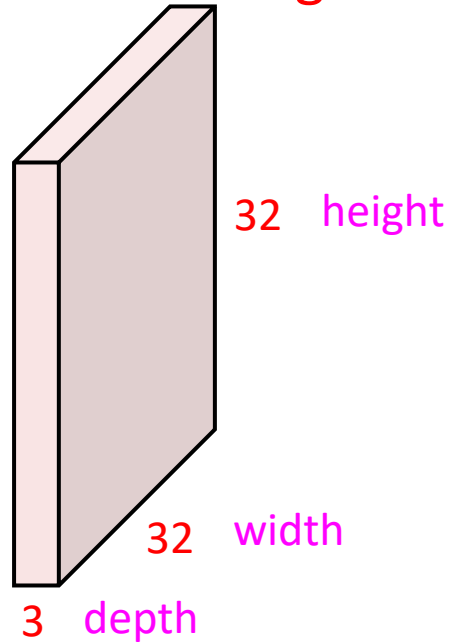


AMA 564 Deep Learning

2026 Spring

Lecture 5

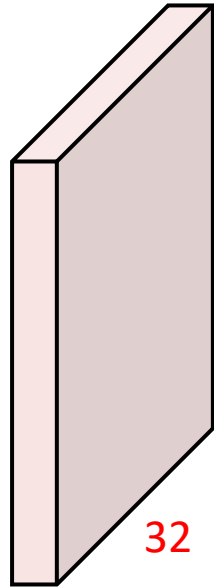
32x32x3 image



CNN Recap: Convolution Layer



32x32x3 image



32 height

32 width

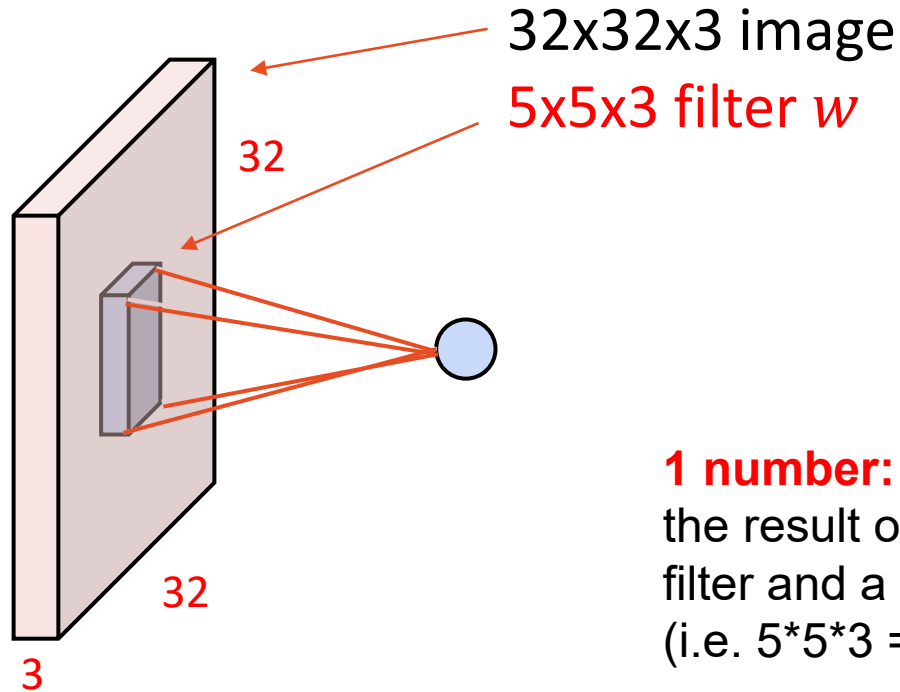
3 depth

Filters always extend the full depth of the input volume

5x5x3 filter



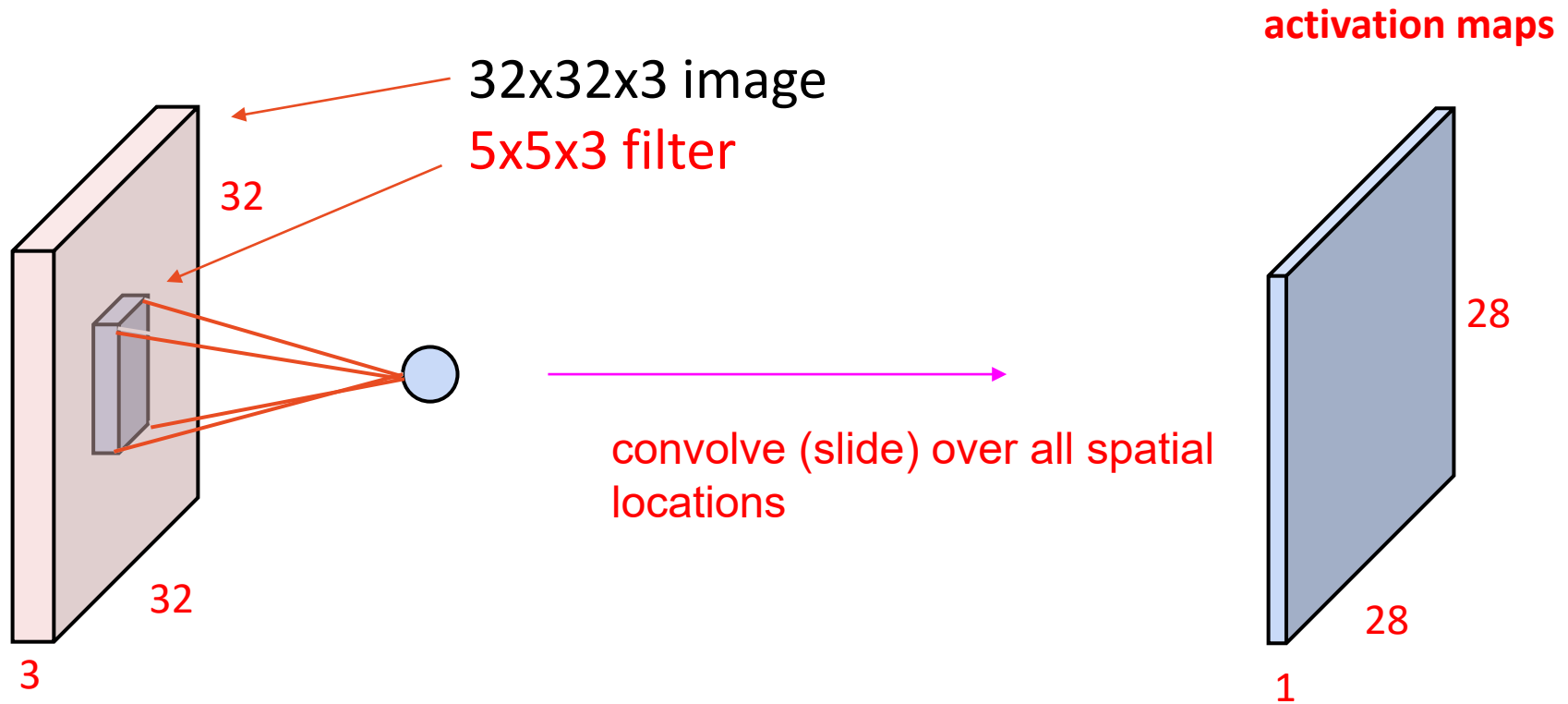
Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

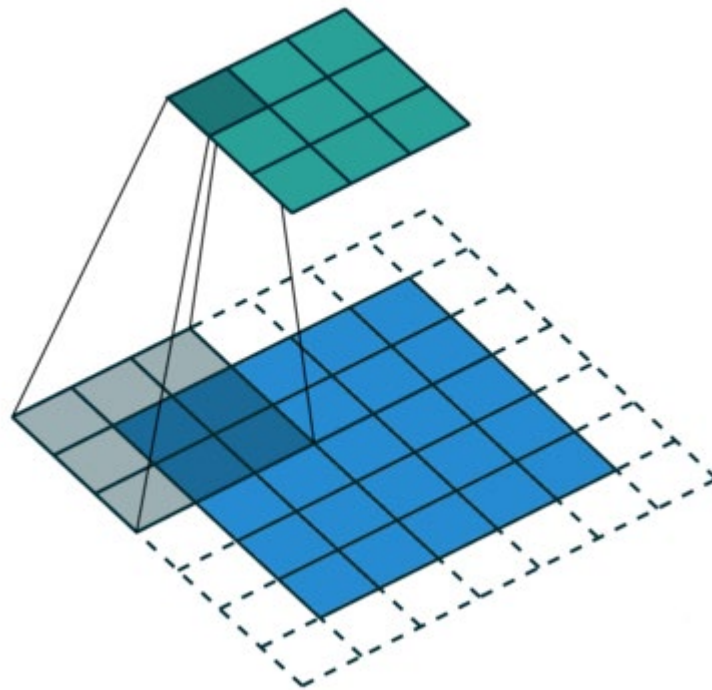


the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. $5*5*3 = 75$ -dimensional dot product + bias)

$$w^T x + b$$

CNN Recap: Convolution Layer



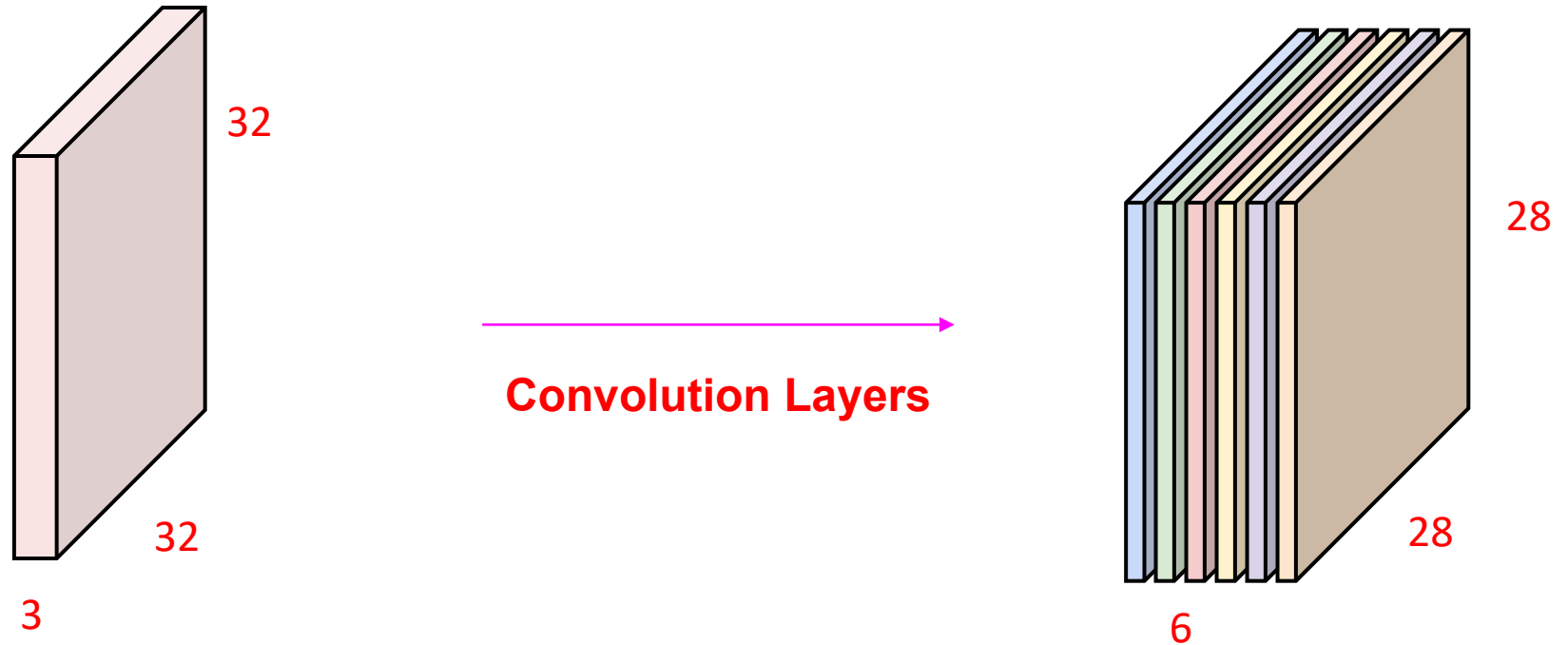


Output

Filter

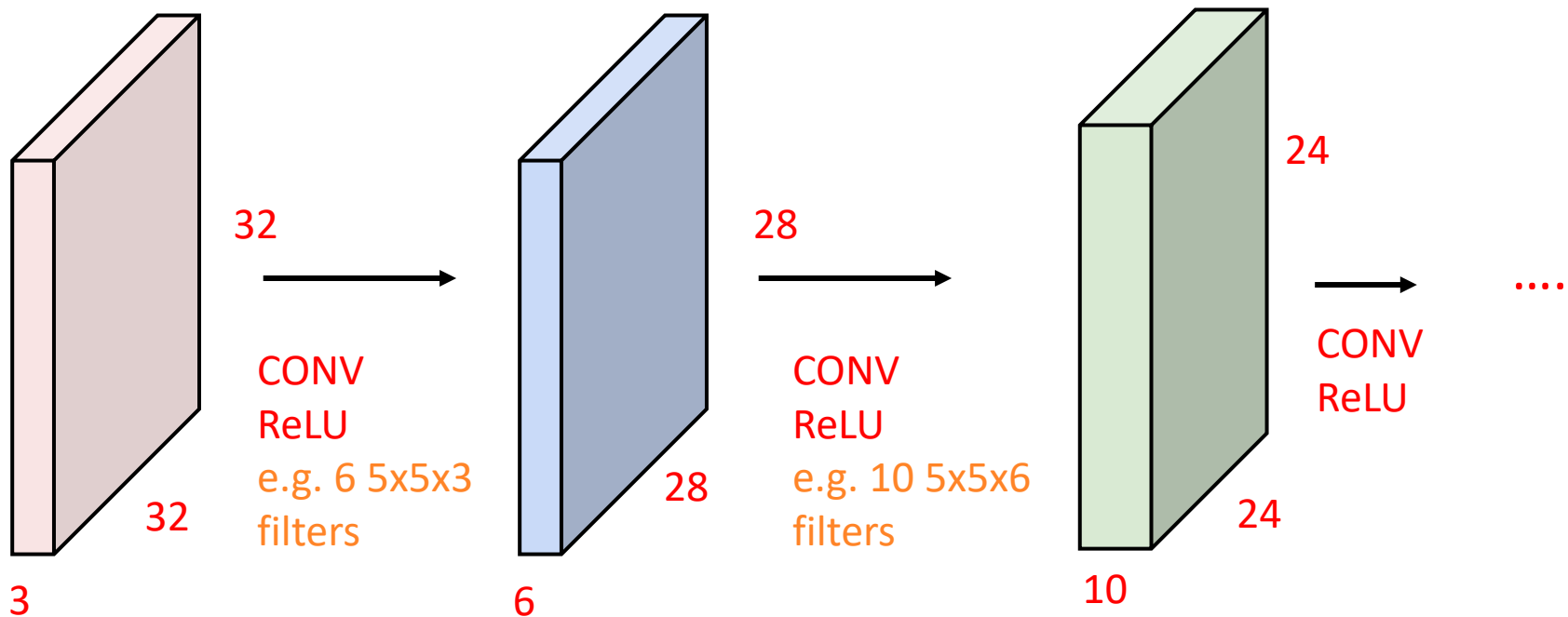
Input

Padding



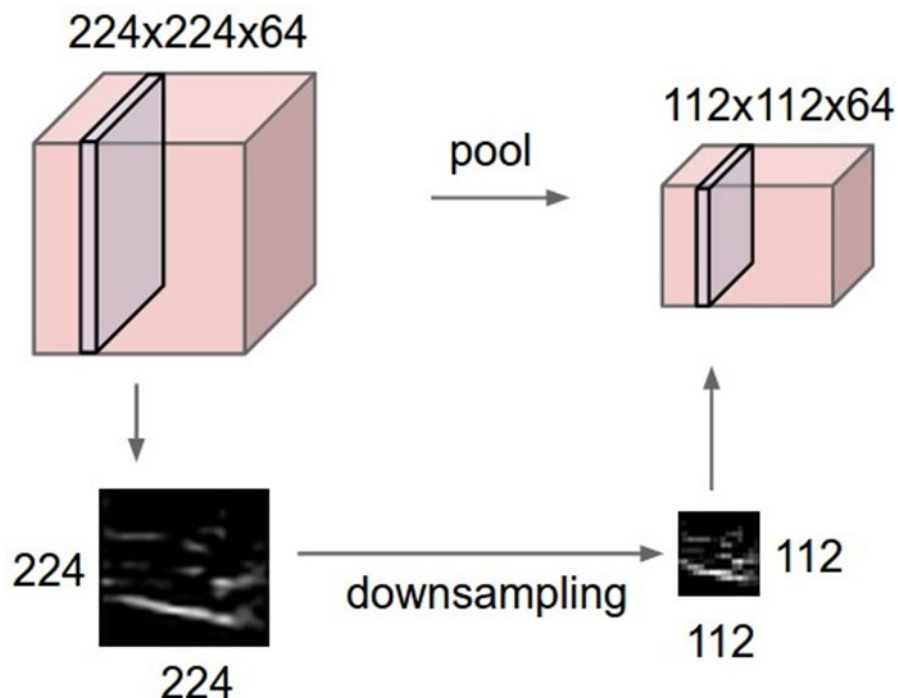
Stack these up to get a new “image” of size 28x28x6!

CNN is a sequence of **Convolution Layers**, interspersed with activation functions.



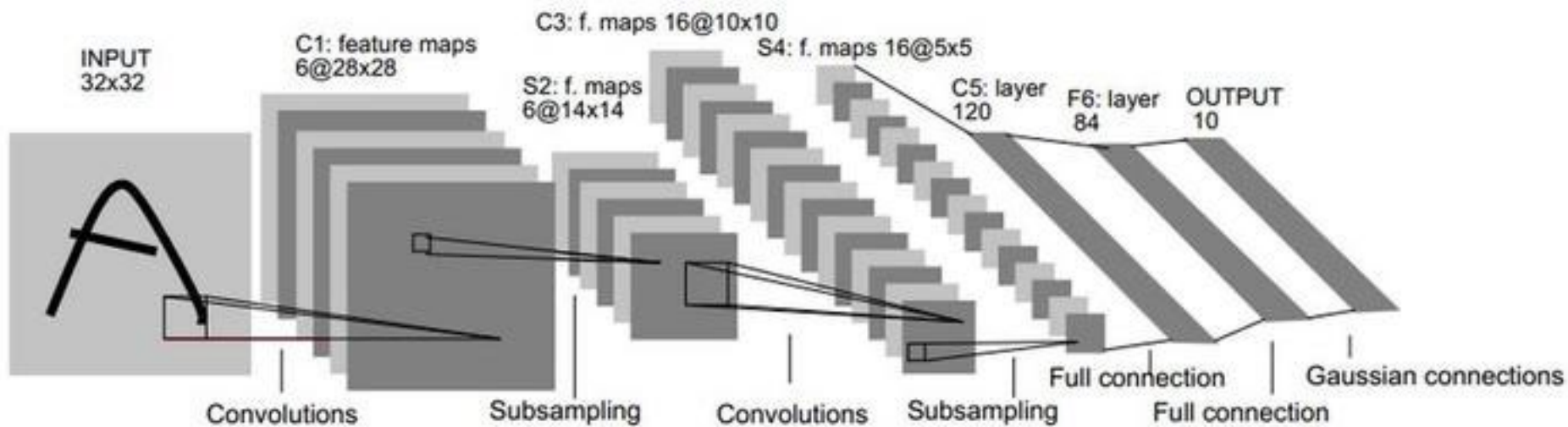
Pooling Layer is a down sampling strategy.

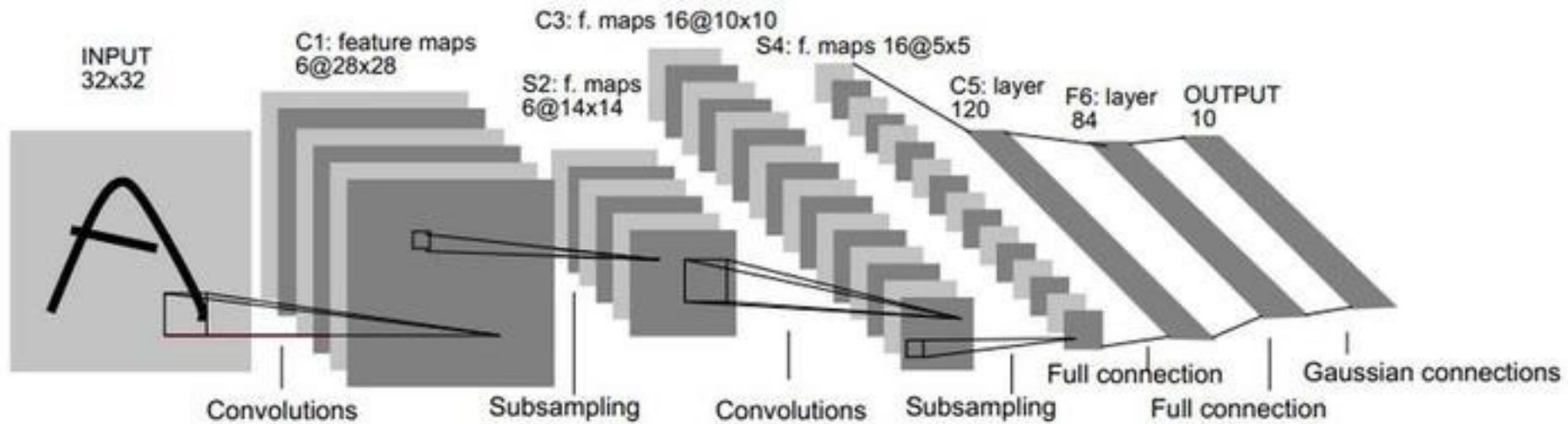
1. Construct better translationally invariant features.
2. Learn more compact features.



Famous CNN Architectures

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.





Key Summary:

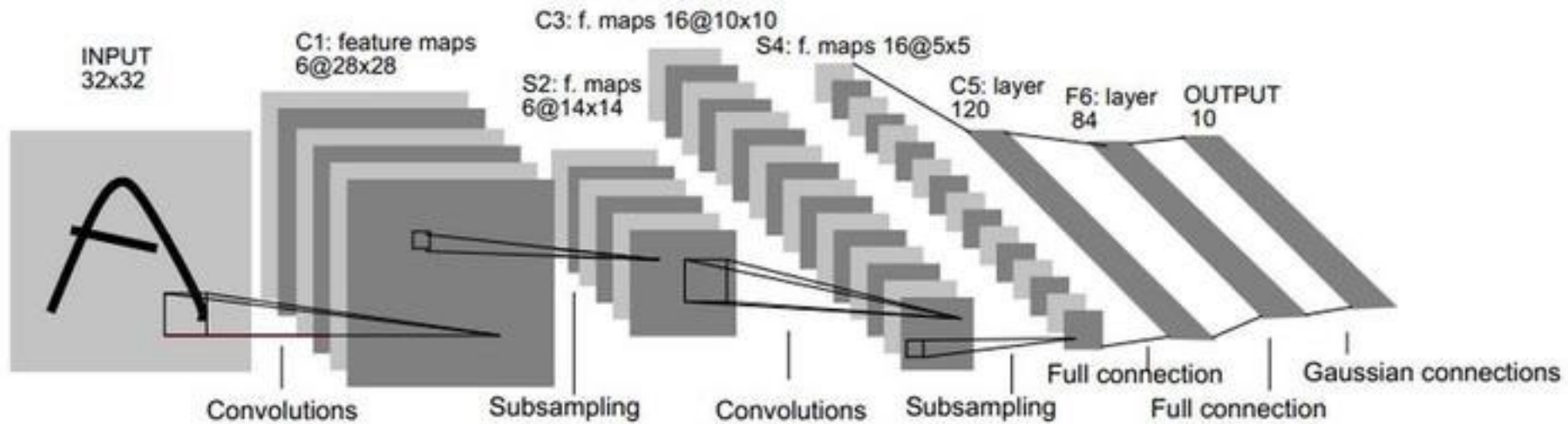
7 Layers (input layer not counted)

3 Convolution Layers (C1; C3; C5)

2 Pooling Layers (S2; S4)

2 Fully Connected Layers (F6; Output)

Sigmoid Activation!



Details:

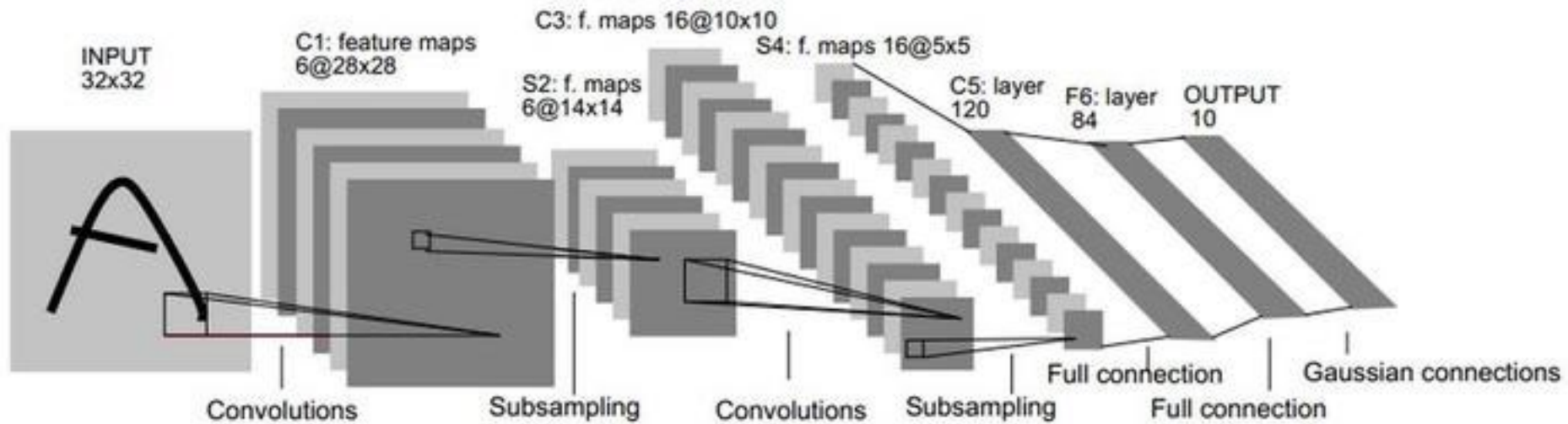
C1: Convolution Layer

Input channel: 1 $\longrightarrow$ Output channel: 6

Input size: 1 x 32 x 32

Kernel size: 5 x 5

Output size: 6 x 28 x 28



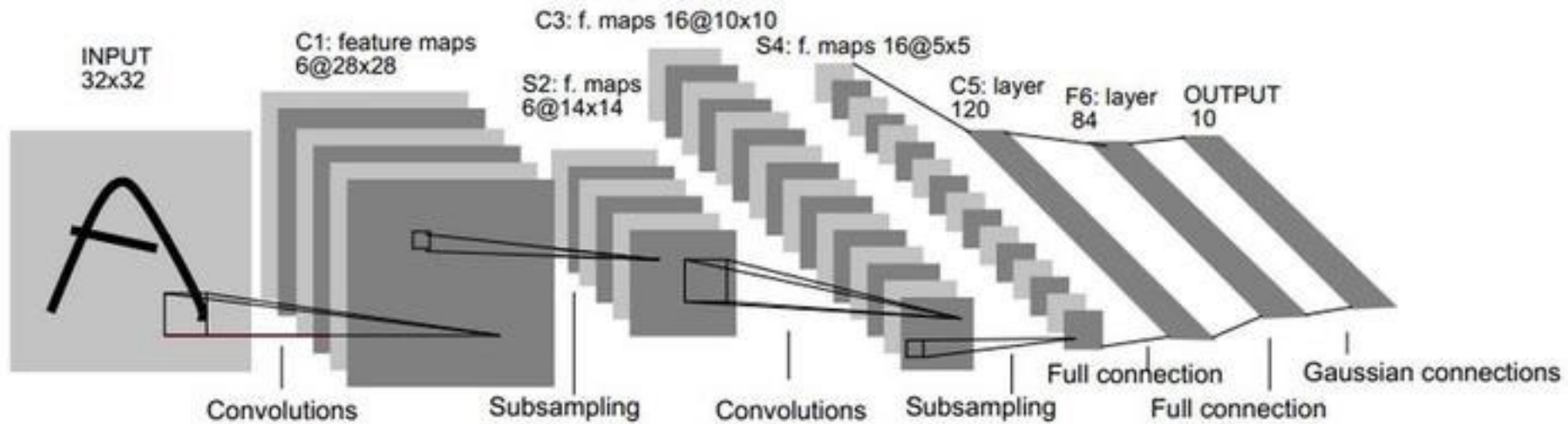
Details:

S2: Max Pooling Layer

Input size: 6 x 28 x 28

Pooling size: 2 x 2

Output size: 6 x 14 x 14



Details:

C3: Convolution Layer

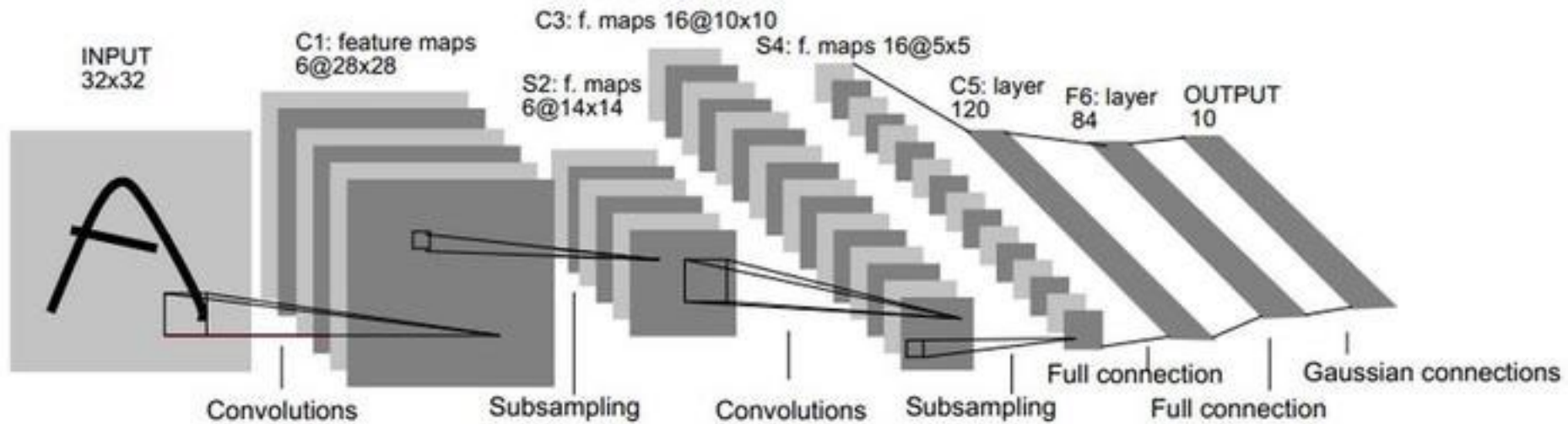
Input channel: 6

Output channel: 16

Input size: 6 x 14 x 14

Kernel size: 5 x 5

Output size: 16 x 10 x 10



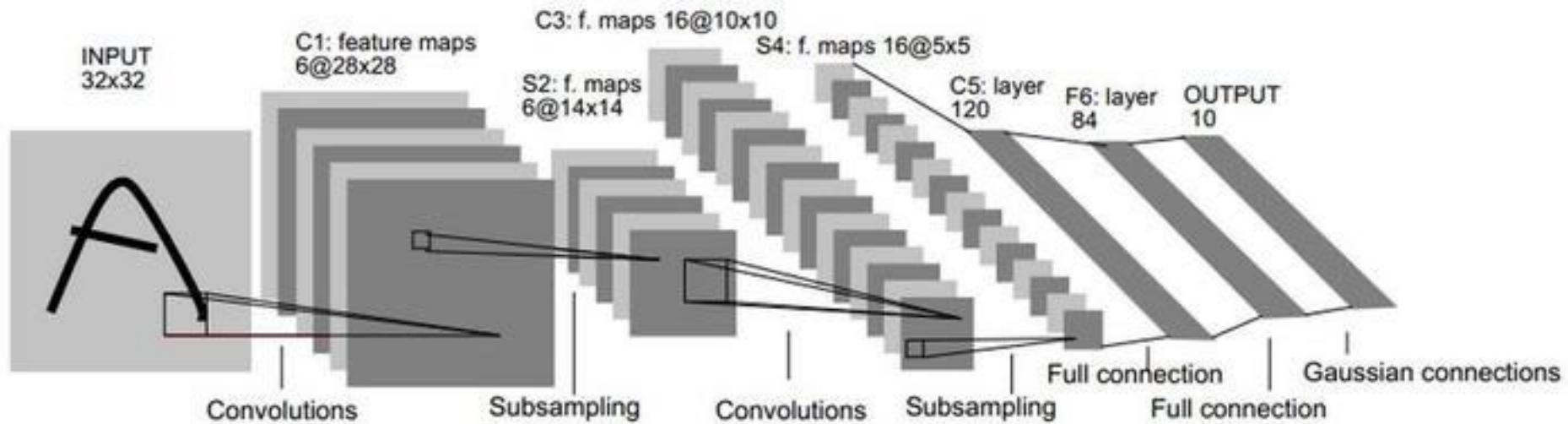
Details:

S4: Max Pooling Layer

Input size: 16 x 10 x 10

Pooling size: 2 x 2

Output size: 16 x 5 x 5



Details:

C5: Convolution Layer

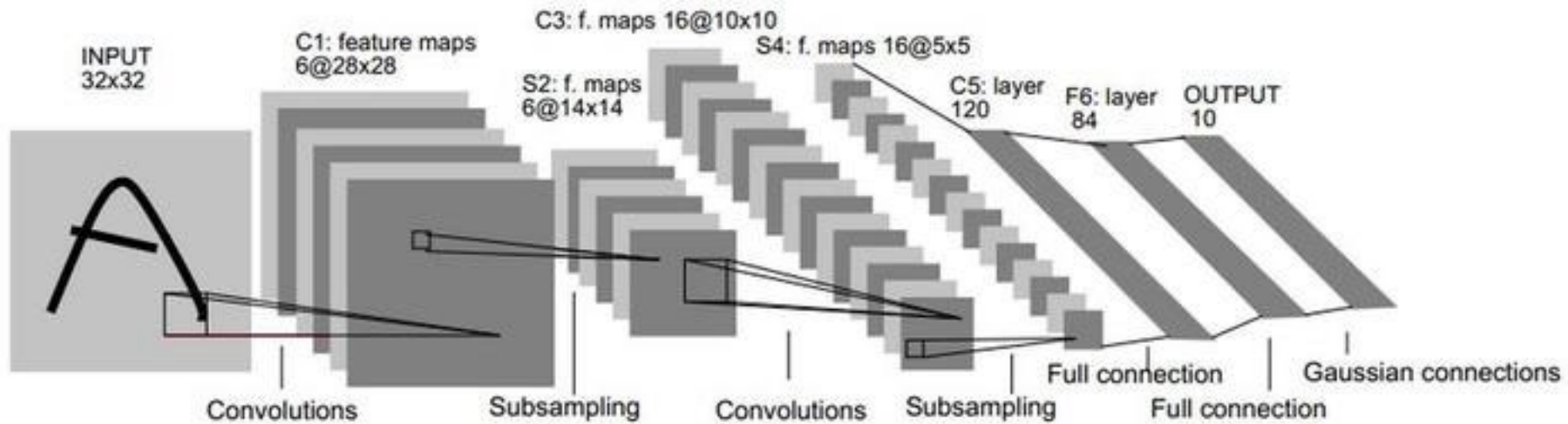
Input channel: 16

Output channel: 120

Input size: 16 x 5 x 5

Kernel size: 5 x 5

Output size: 1 x 120

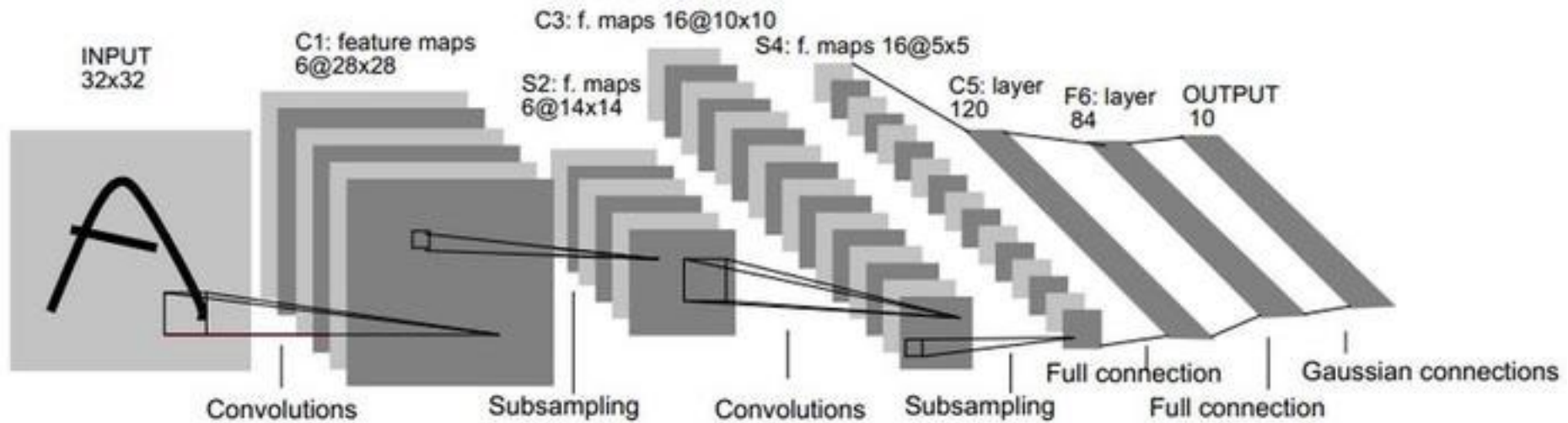


Details:

F6: Fully Connected Layer

Input size: 1 x 120

Output size: 1 x 84

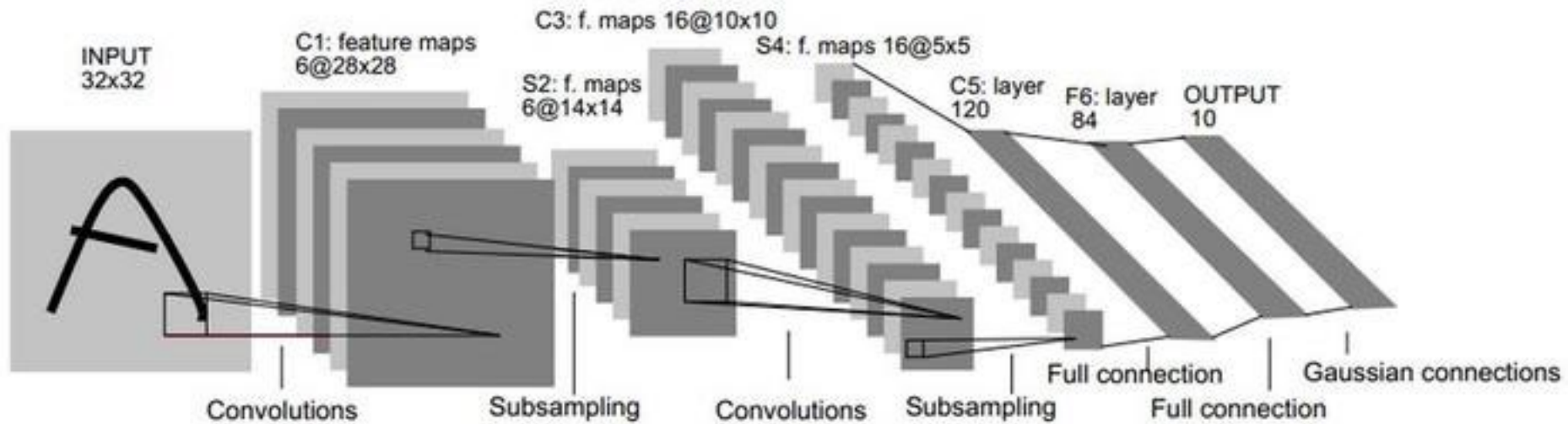


Details:

Output: Fully Connected Layer

Input size: 1 x 84

Output size: 1 x 10



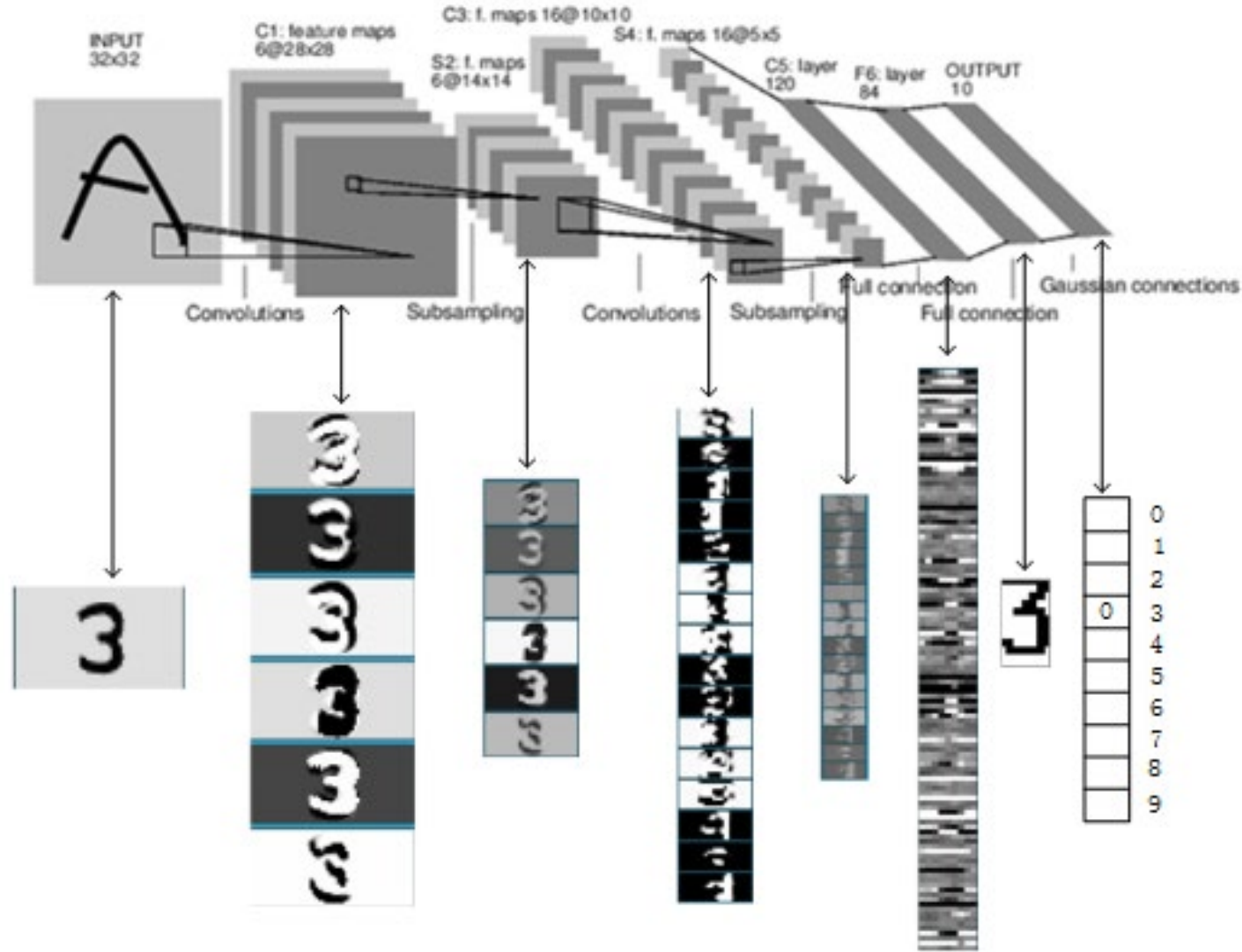
Summary Remark:

A Simple CNN architecture.

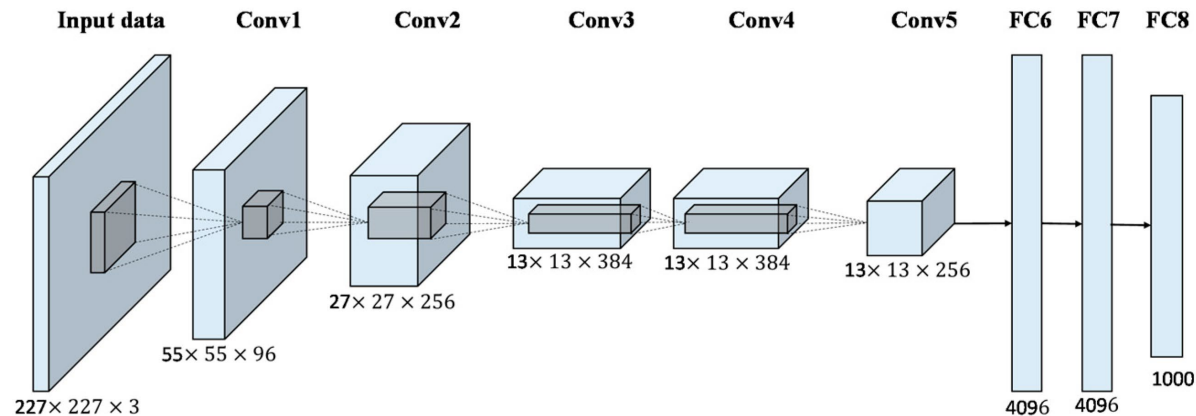
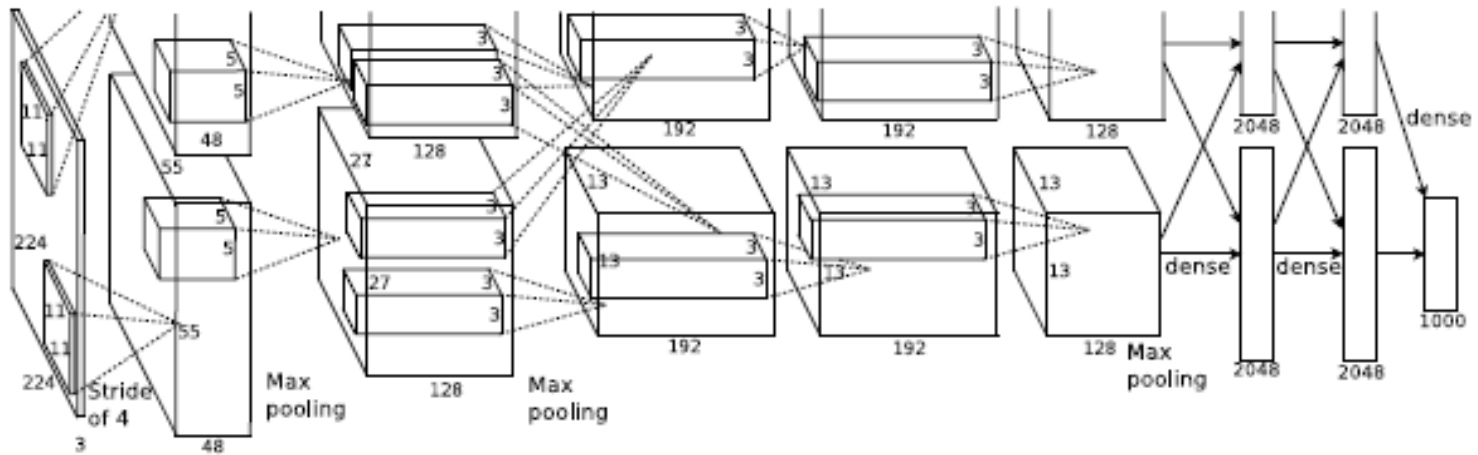
Use **Sigmoid** as activation function.

Convolution layer can convert a tensor to a vector!

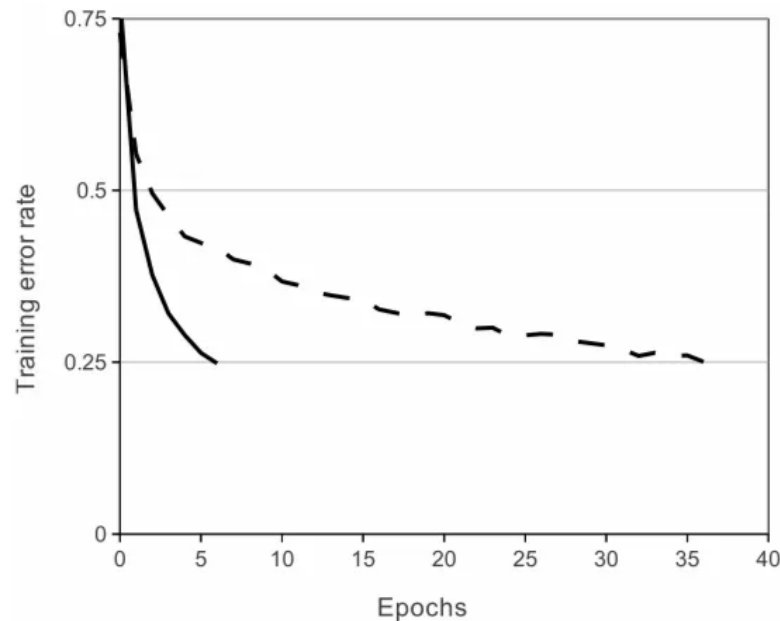
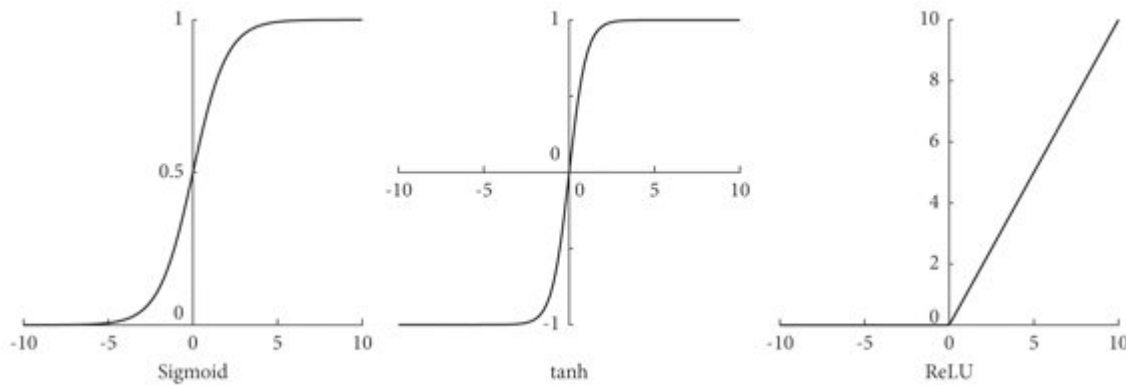
A Visualization with Image “3”



Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *NIPS*.



Feature #1 of AlexNet: Using ReLu Activation

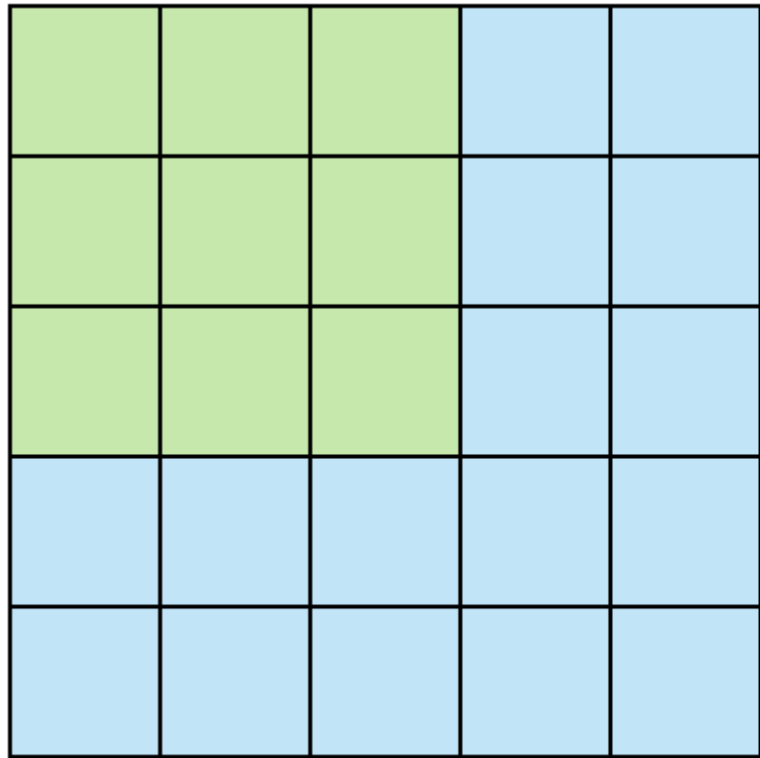


CNNs that use ReLU achieved a 25% error rate on CIFAR-10 is **six times** faster than those that used tanh.

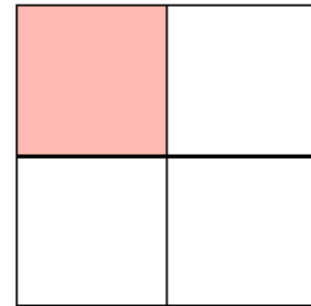
Feature #2 of AlexNet: Overlapping Pooling



AlexNet uses **3 x 3 max pooling** with **stride 2**.



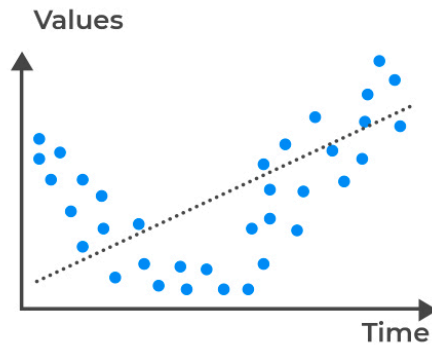
Stride 2



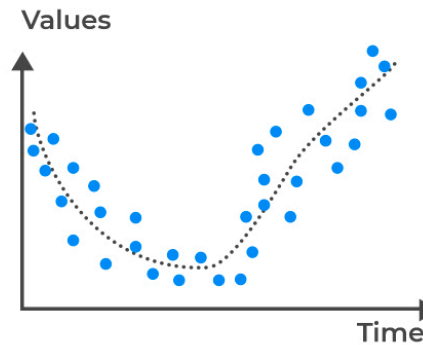
Feature Map



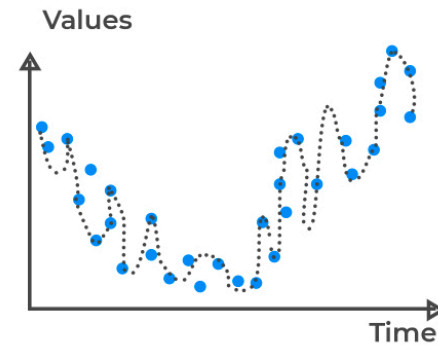
Generalization and Overfitting



Underfitted
(High bias error)



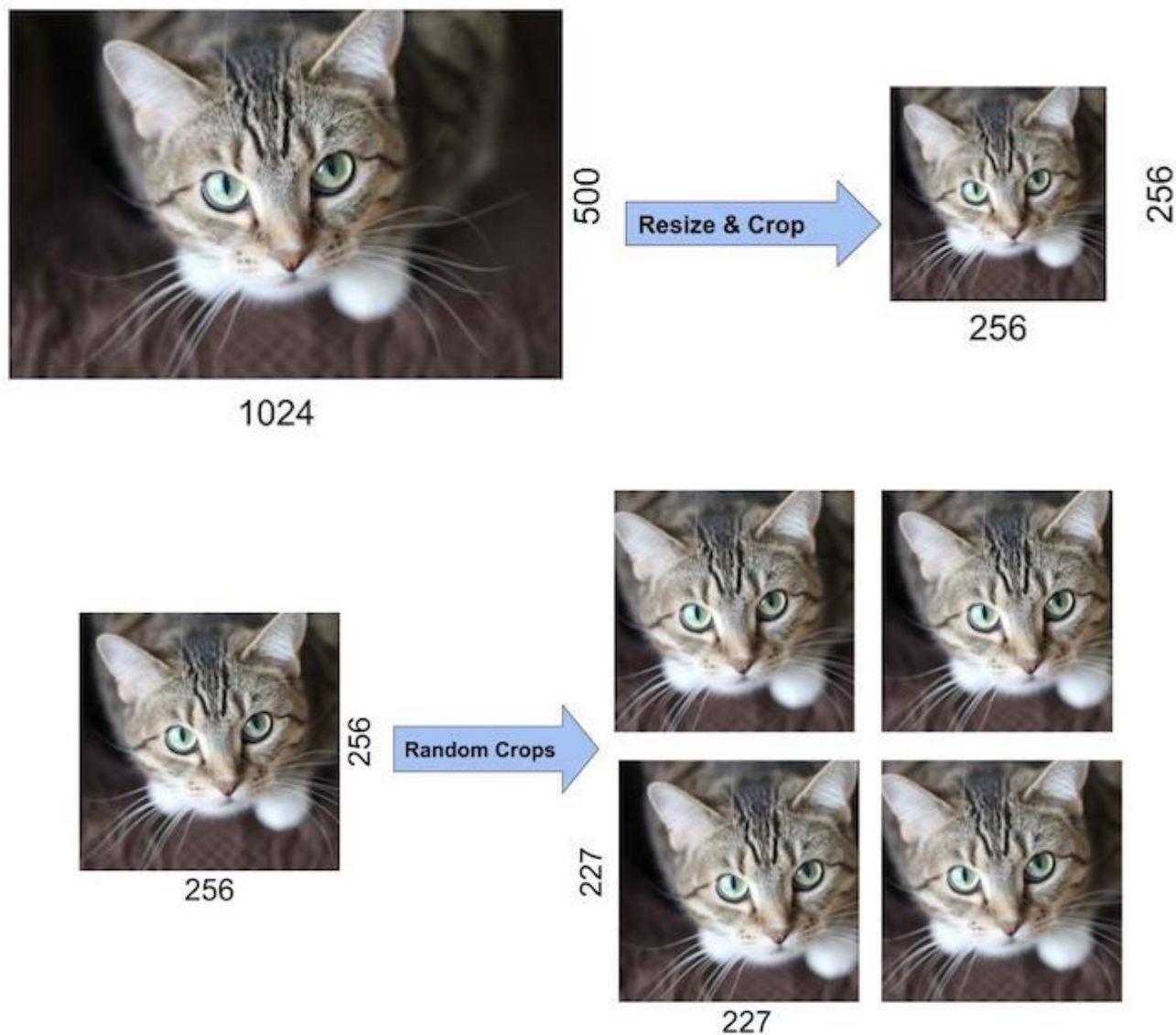
Good Fit/Robust
(Balance between
bias and variance)



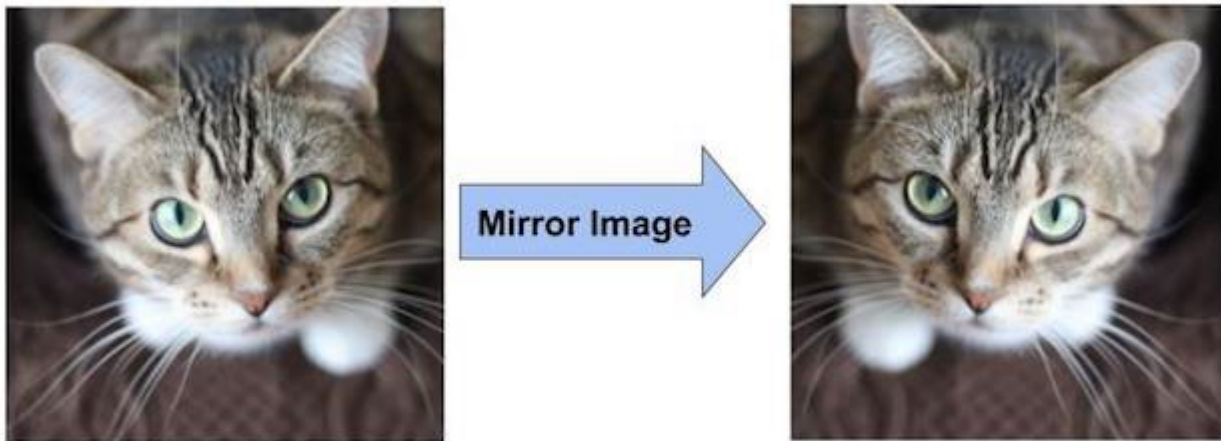
Overfitted
(High variance error)

A highly complex model may not be overfitting if (1) A large and complex dataset is available; (2) a large number of model parameters are **zero**!!!

Feature #3 of AlexNet: Data Augmentation



Feature #3 of AlexNet: Data Augmentation

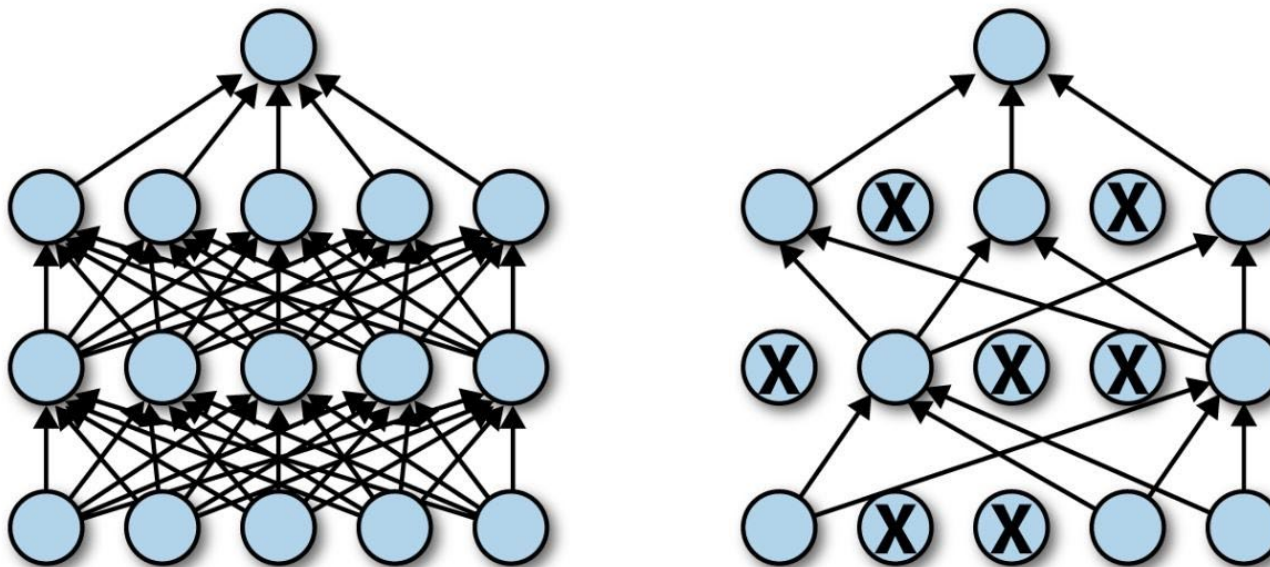




Rotation



Feature #4 of AlexNet: Dropout



A **dropout layer** randomly sets input elements to zero with a given probability.

AlexNet uses dropout layers with a probability 0.5.

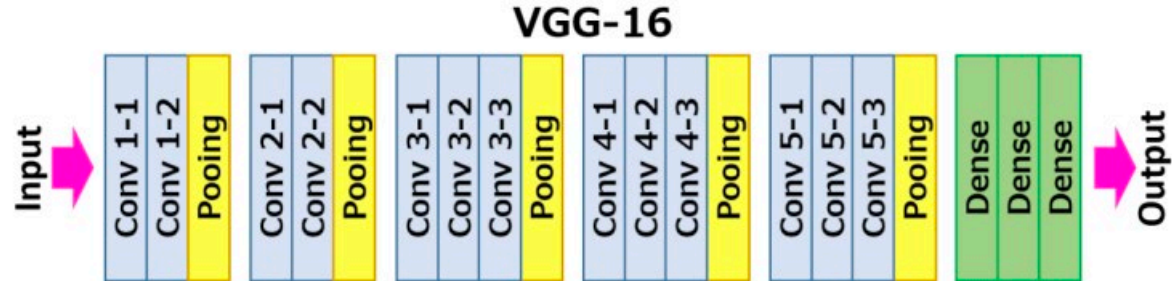
Why does dropout work?

<https://www.youtube.com/watch?v=ARq74QuavAo>



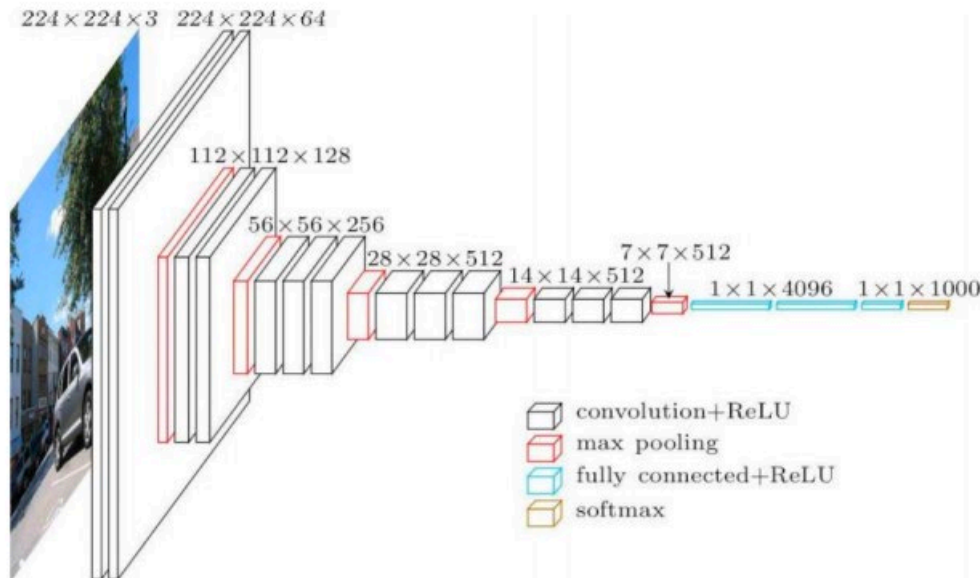
	Top 1 Error	Top 5 Error
Before AlexNet	47.1%	28.2%
AlexNet	37.5%	17%

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *ICLR 2015*.



The Architecture

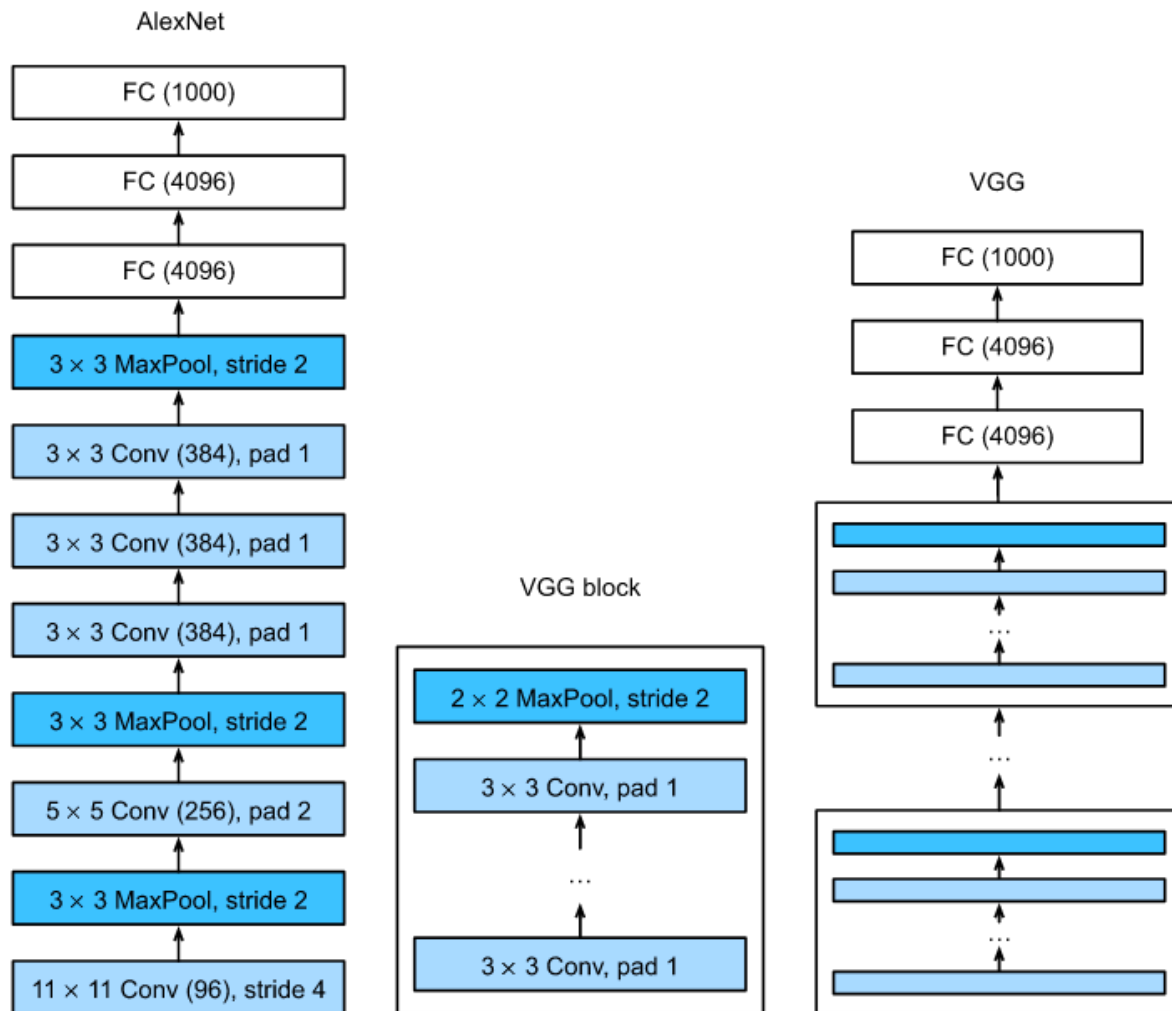
The architecture depicted below is VGG16.



VGG Key: An Evidence to Show Deeper Is Better



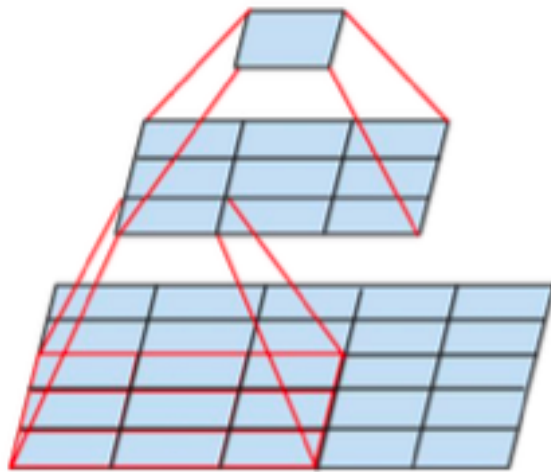
Key Idea of VGG: Replace the large convolution filter by stacking some smaller convolution filters.



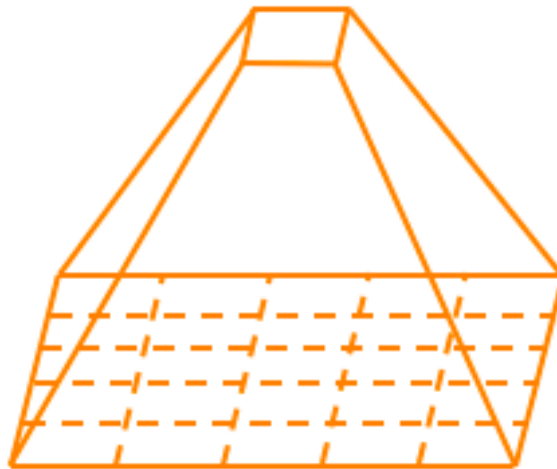
Why smaller convolution filters work?



An example: Replace a 5 x 5 filter with two 3 x 3 filters.



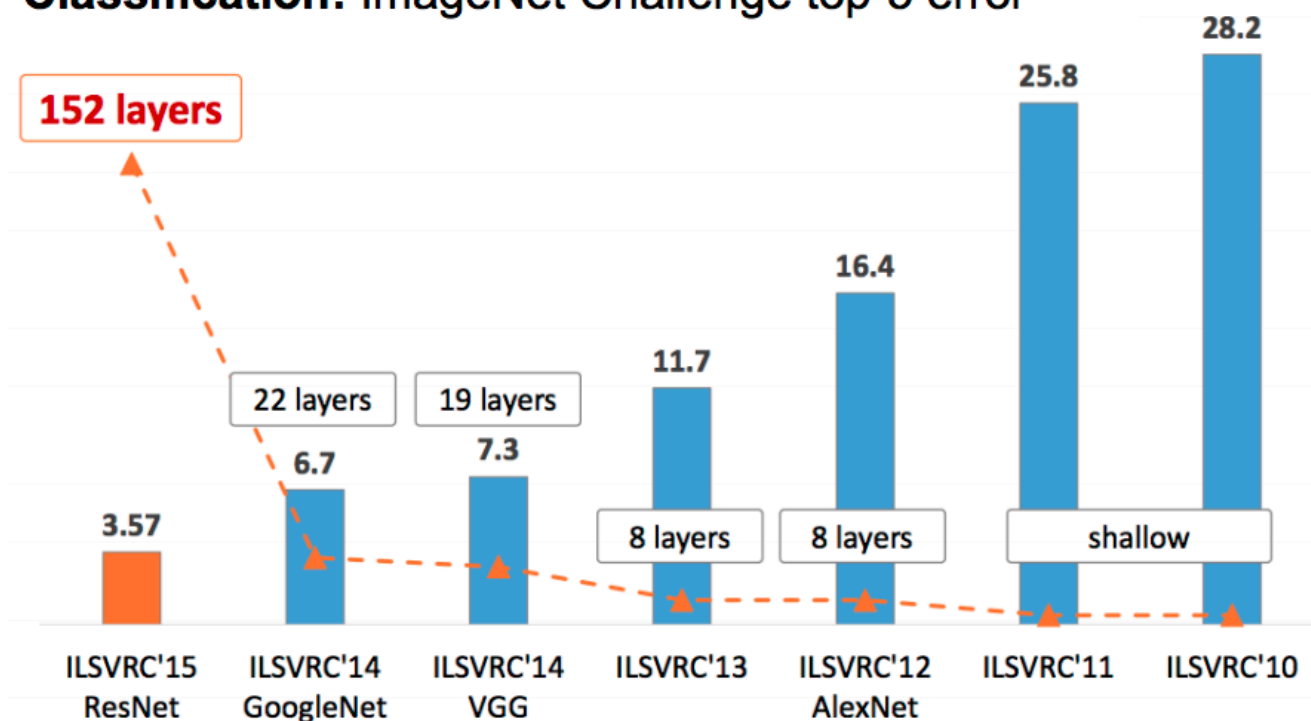
two successive
3x3 convolutions



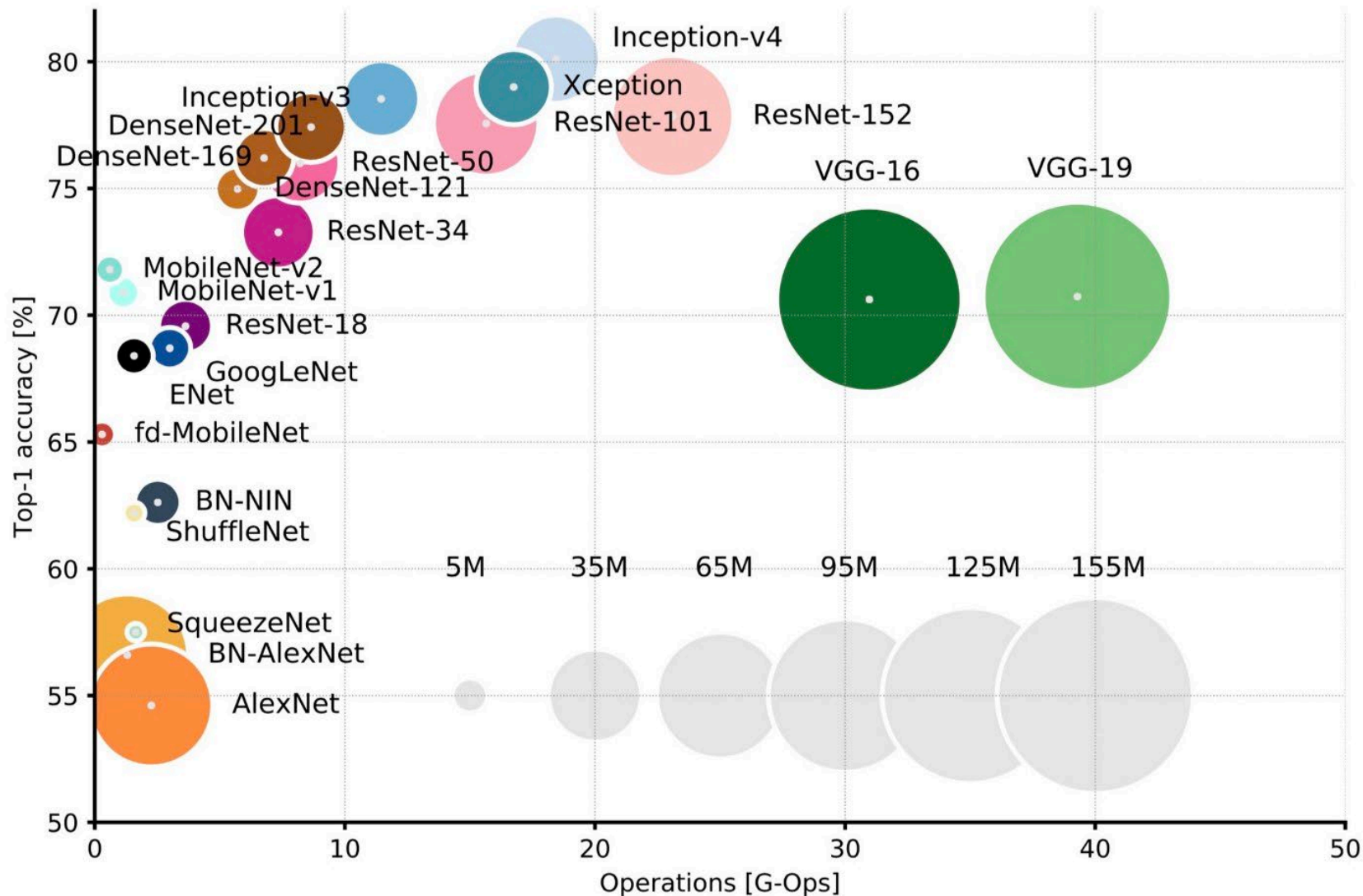
5x5 convolution

1. **More concise and generalizable.**
2. **Smaller filters can achieve better performance than a larger filter.**
3. **Demonstrate that increase depth can boost performance.**

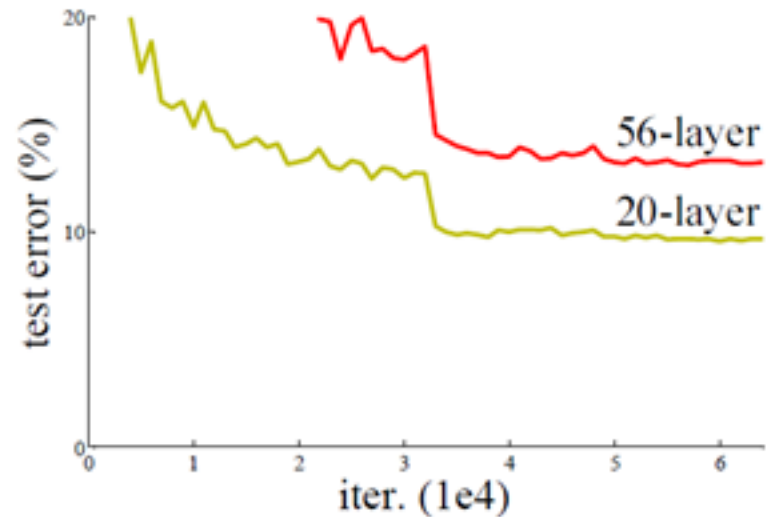
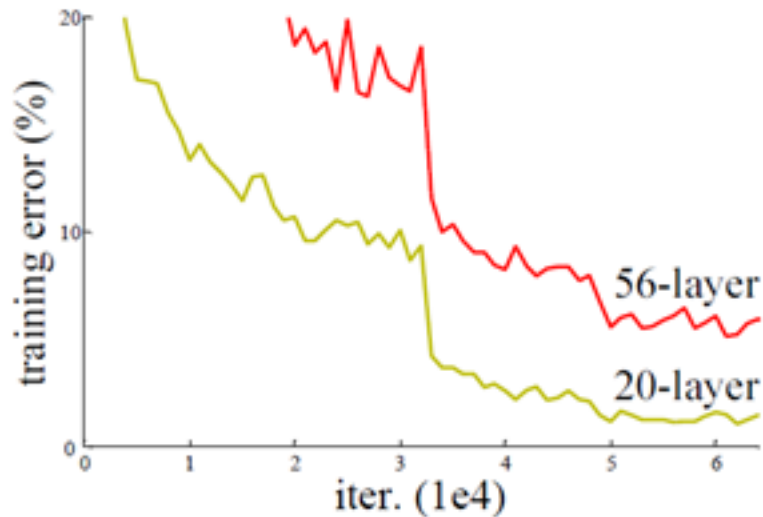
Classification: ImageNet Challenge top-5 error



Disadvantages: Memory Expensive

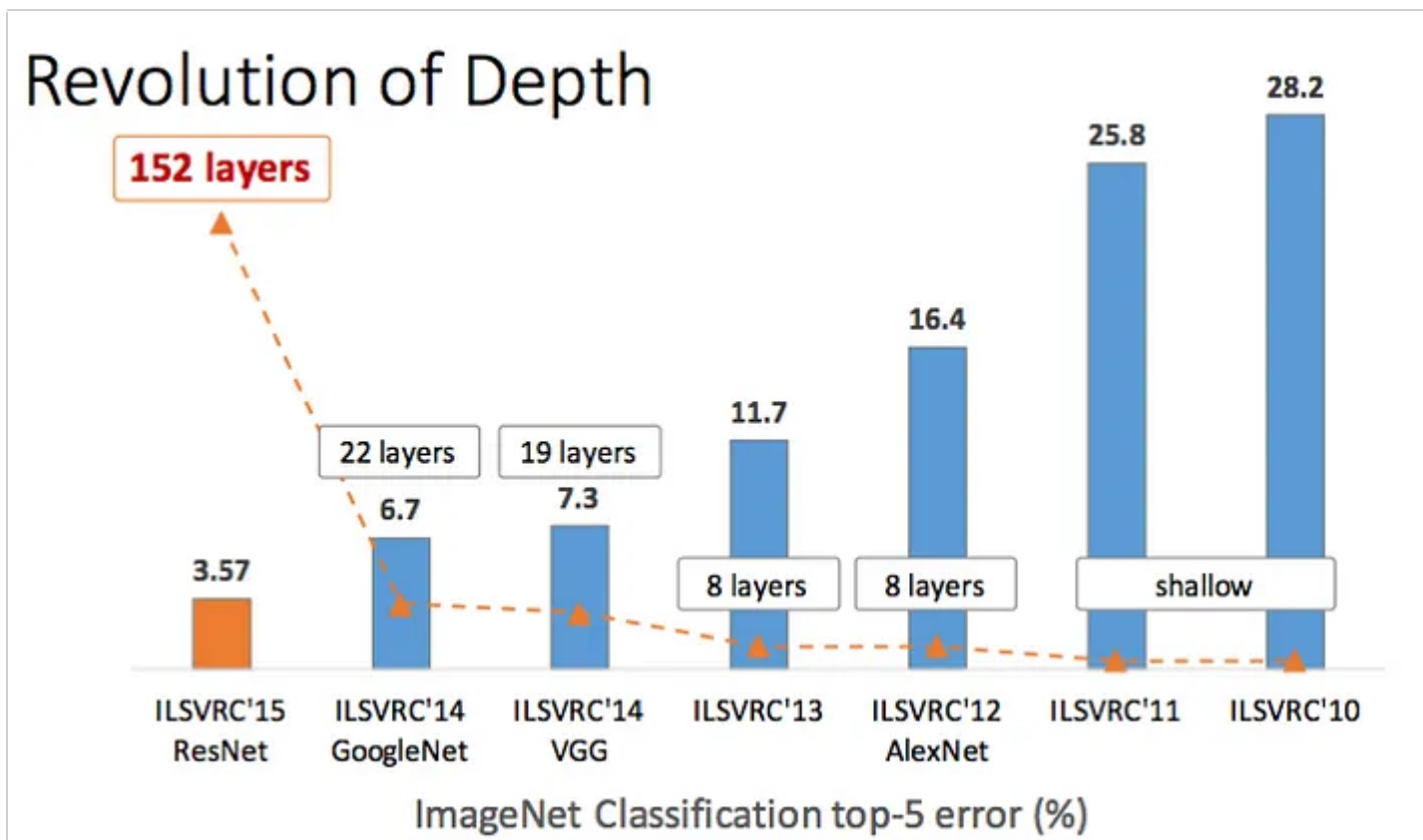


Deeper is Better?



Deep model may be difficult to optimize.

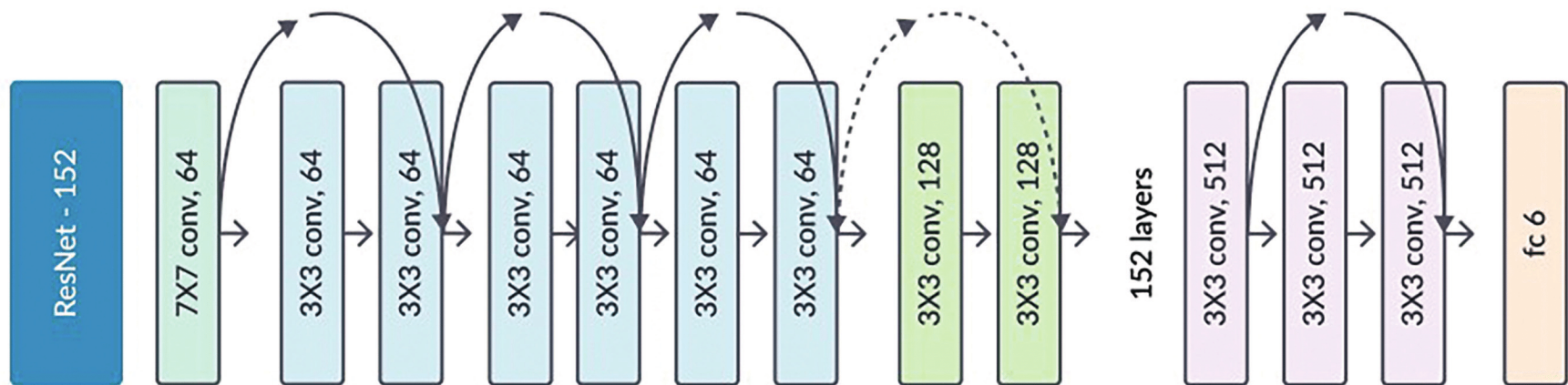
Intuitive sense: Deep model should be at least as good as shallow ones?



A First Glance at ResNet



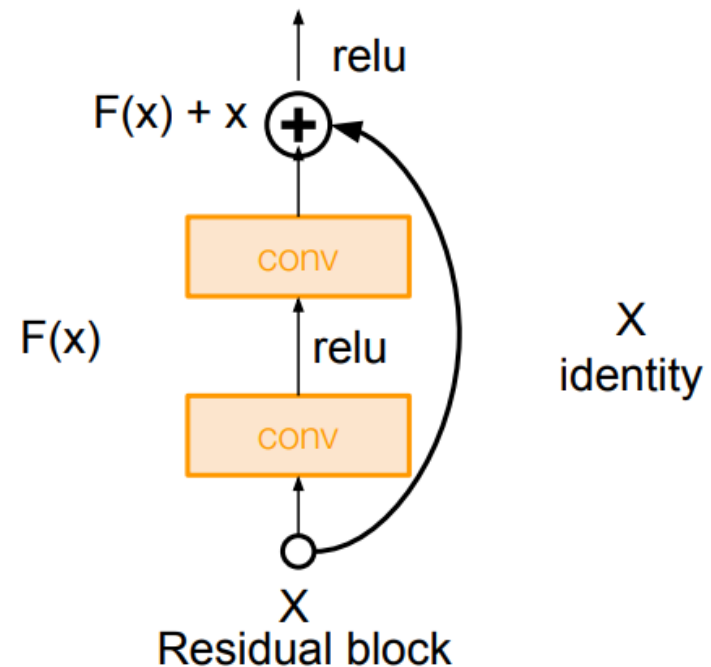
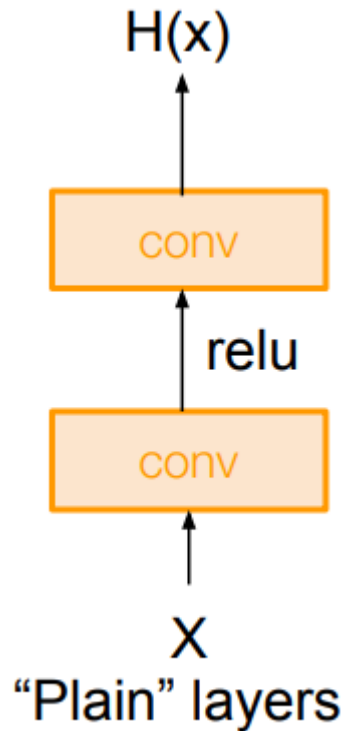
He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. CVPR.



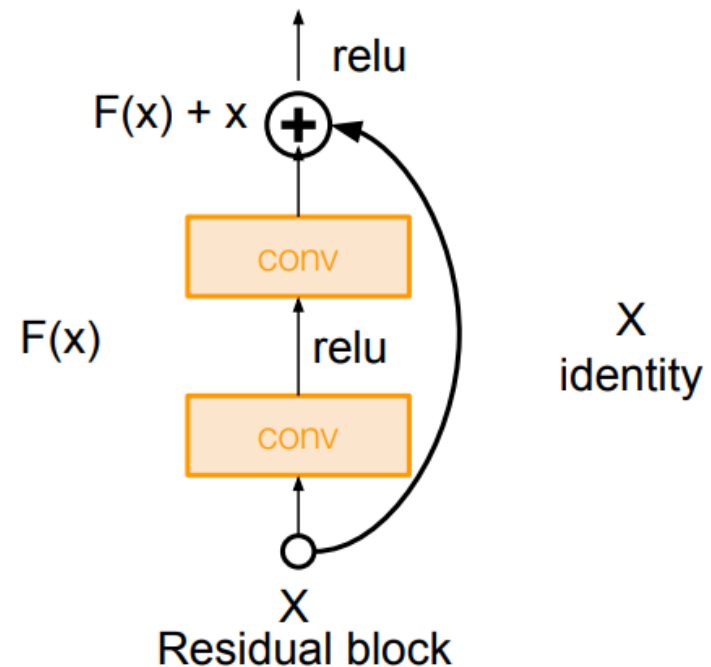
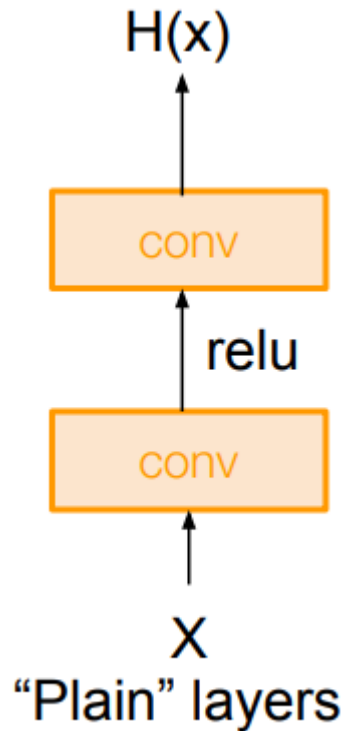
Key numbers: 152 layers for ImageNet.

3.57% Top 5 Error!!!

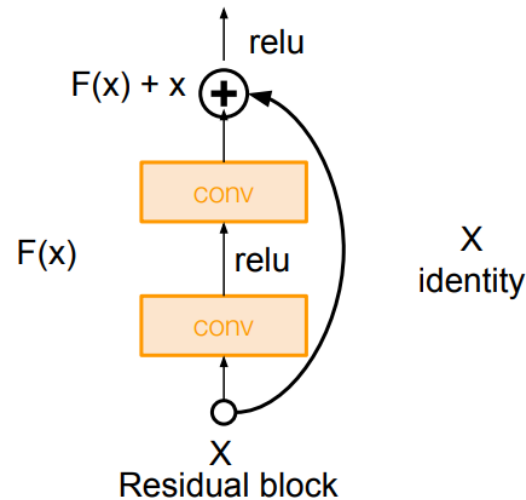
A Key Motivation: It is not easy to learn an identity mapping $f(x) = x$.



A Key Motivation: It is not easy to learn an identity mapping $f(x) = x$.



An identity mapping can now be obtained if $F = 0$.



In a plain network, if we want to learn $H(x)$, now, we only need to learn its residual:

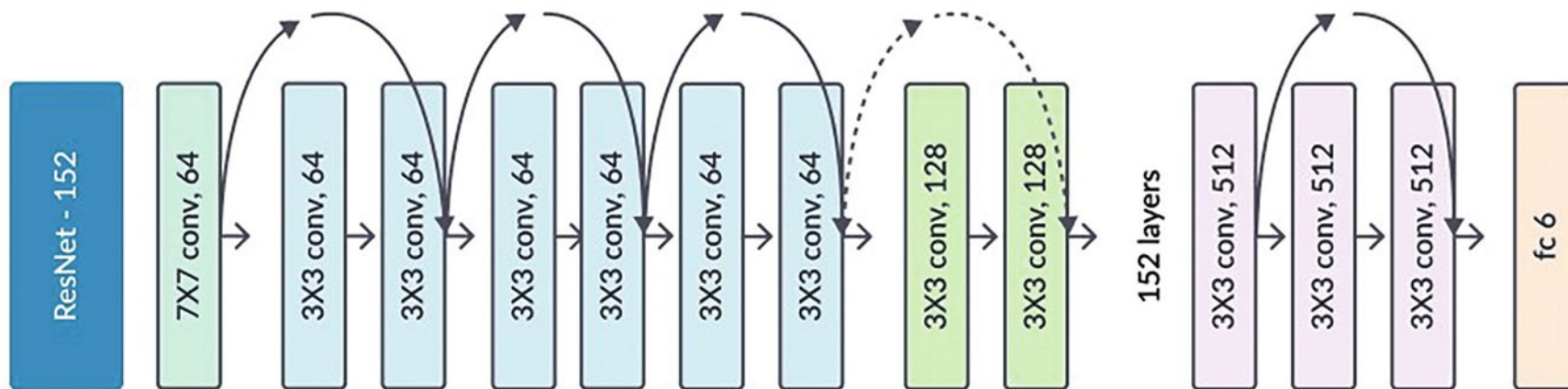
$$F(x) = H(x) - x$$

A stack of three types of residual blocks:

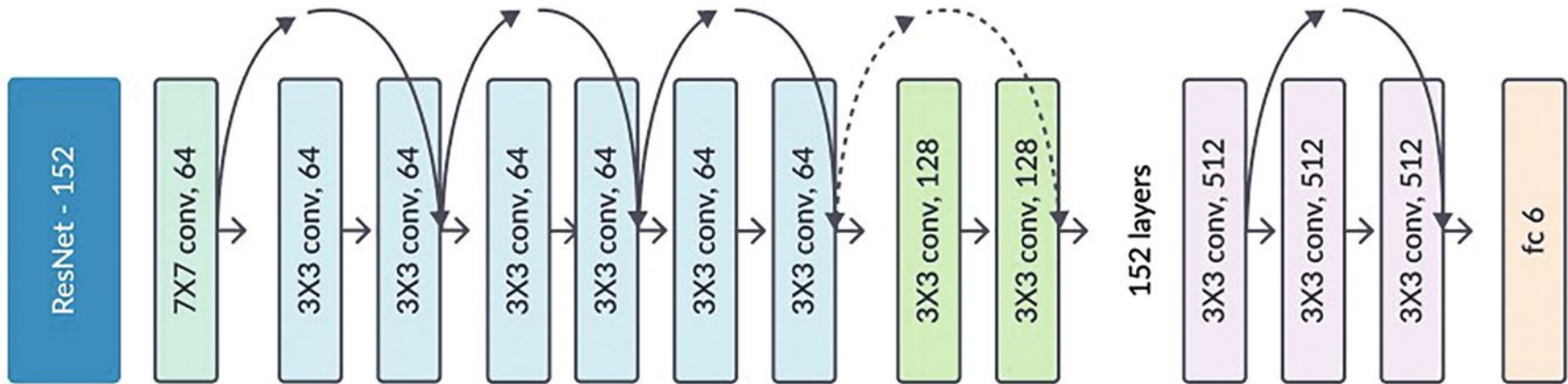
Stage 1: 3x3 convolution filters with **64 channels**.

Stage 2: 3x3 convolution filters with **128 channels**

Stage 3: 3x3 convolution filters with **512 channels**



Key Features of ResNet: Residual Blocks

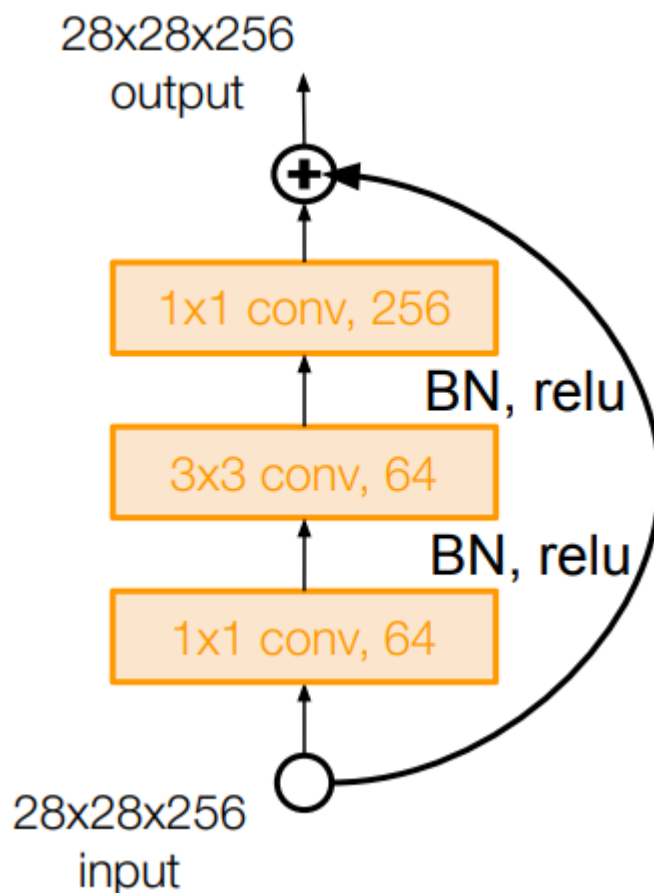


No fully connected layers before output.

Deep ResNet: Use “Bottleneck” layer



For very deep neural networks (i.e., 50+ layers), we can use “**bottleneck**” building blocks to improve efficiency.

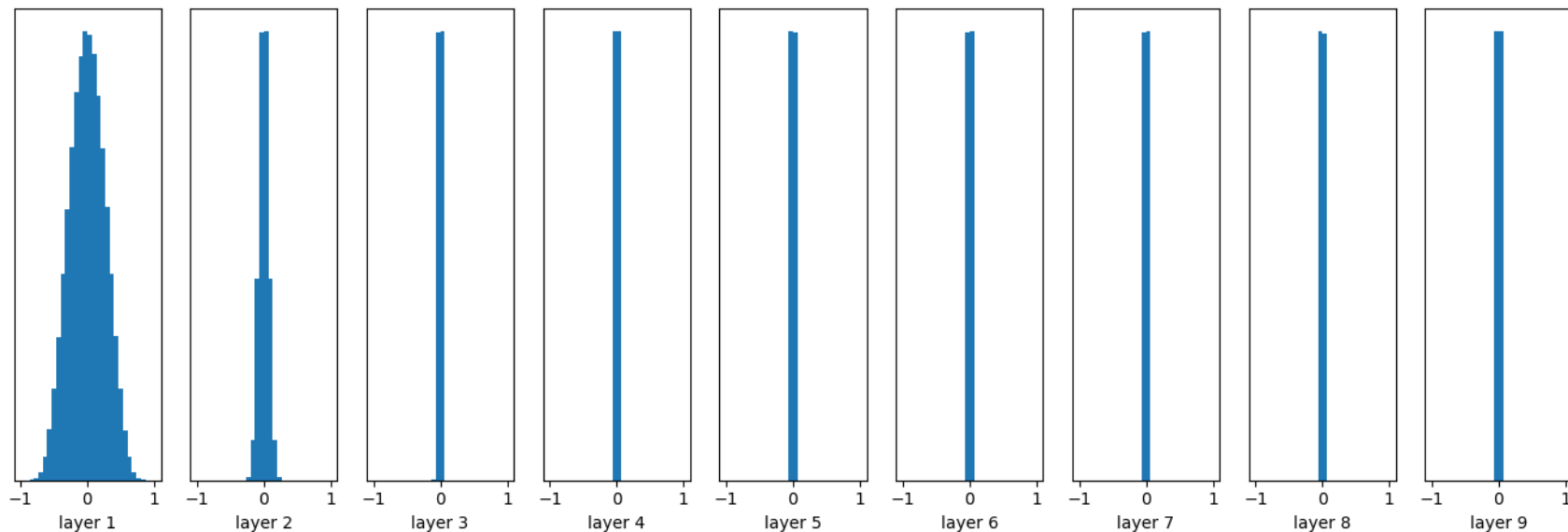


- Initialization using He's Xavier initialization.
- Batch normalization after each convolution layer.
- SGD + Momentum (0.9)
- Dynamic learning rate scheduling.
- No dropout used in He's ResNet paper.

Weight Initialization: Is Random Initialization Good?



Consider a 10-layer DNN with tanh activation function. If we initialize all the weights with normal distribution $N(0, 0.01)$.

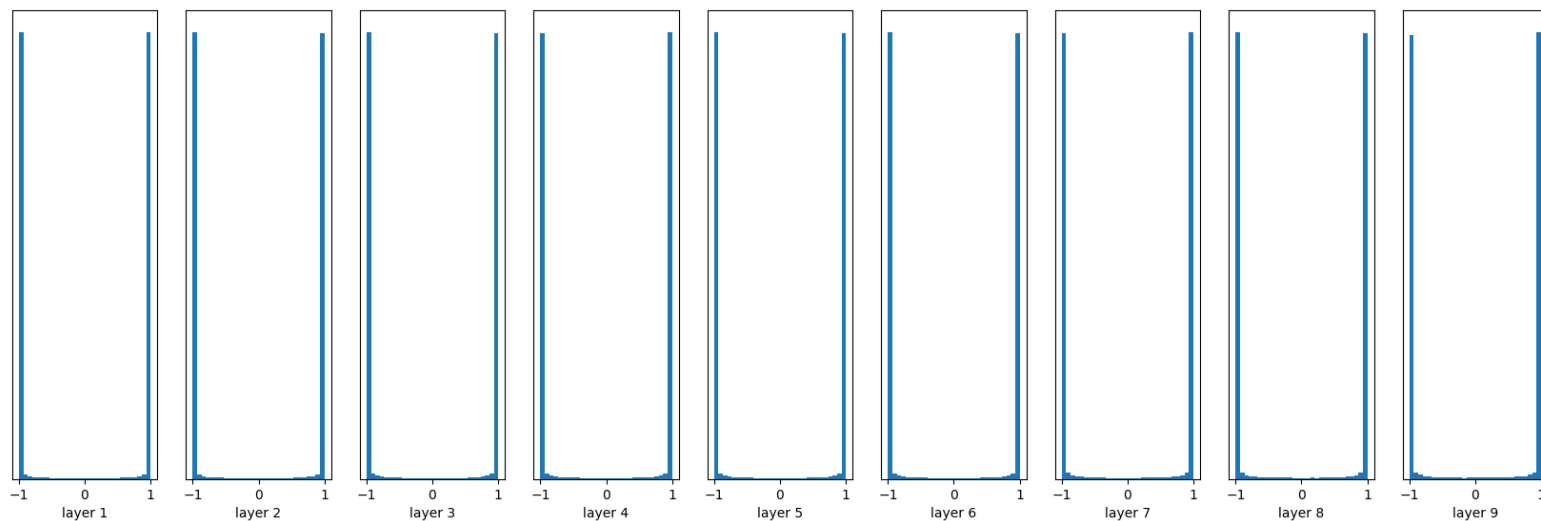


The outputs of the last a few layers are almost zeros.



Weight Initialization: Is Random Initialization Good?

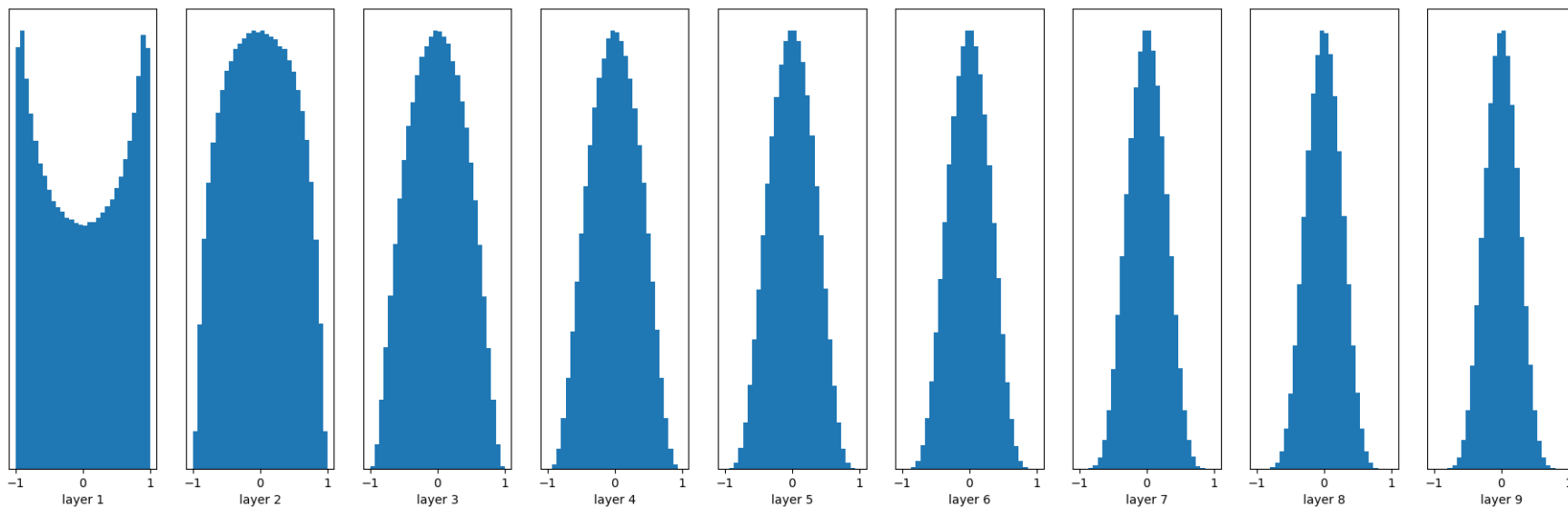
Consider a 10-layer DNN with tanh activation function. If we initialize all the weights with normal distribution $N(0, 1)$.



The outputs are close to -1 or 1, the gradients will be very small for tanh activation function.

Key Motivation: Keep a same variance for the input and the output.

Consider a 10-layer DNN with tanh activation function. If we initialize all the weights with normal distribution $N(0, 1)/\sqrt{n_{in}}$.



A derivation of Xavier Initialization



Consider $y = w_1x_1 + w_2x_2 + \dots + w_nx_n$, x_i are i.i.d. with zero mean, w_i are i.i.d with zero mean.

Target: Compute $Var[y]$.

Lemma $Var[w_ix_i] = (E[w_i])^2Var[x_i] + (E[x_i])^2Var[w_i] + Var[w_i]Var[x_i]$.

Thus, $Var[w_ix_i] = Var[w_i]Var[x_i]$ and

$$Var[y] = Var[w_1x_1 + w_2x_2 + \dots + w_nx_n] = \sum_{i=1}^n Var[w_ix_i] = nVar[w_i]Var[x_i]$$

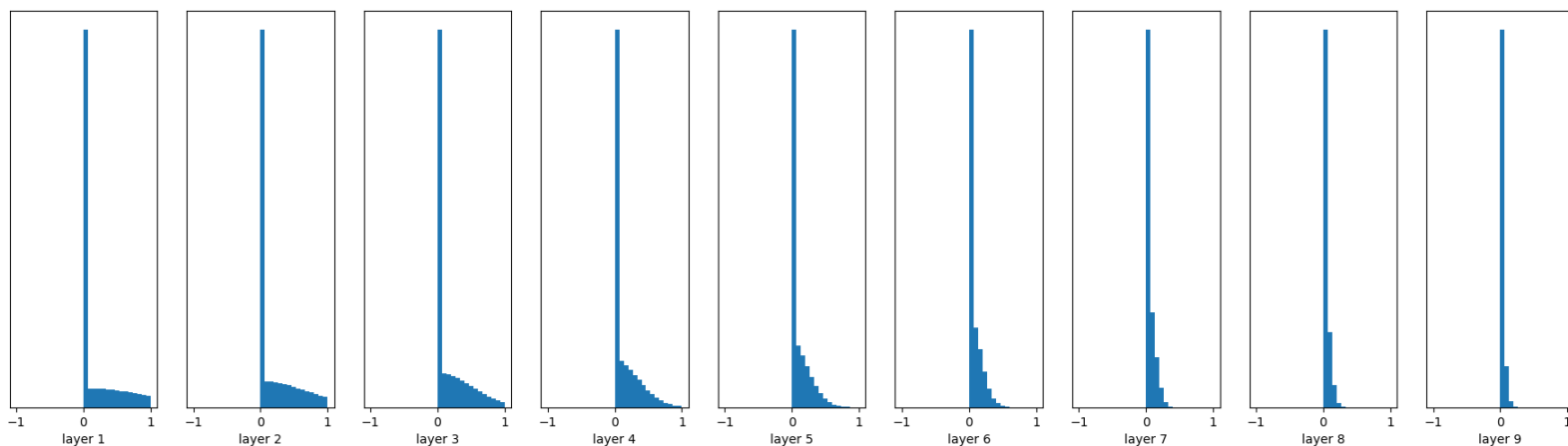
Thus,

$$Var[w_i] = 1/n$$



How about for ReLu Activation Function?

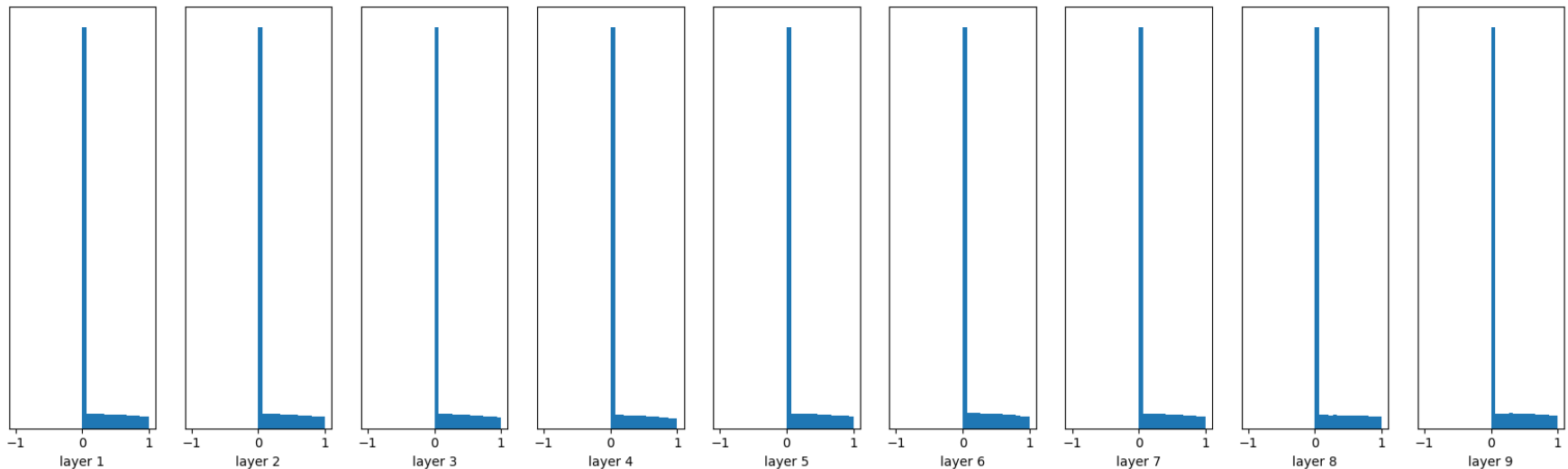
Consider a 10-layer DNN with **ReLU** activation function. If we initialize all the weights with normal distribution $N(0, 1)/\text{sqrt}(n_{\text{in}})$.



Not that good for the last a few layers.

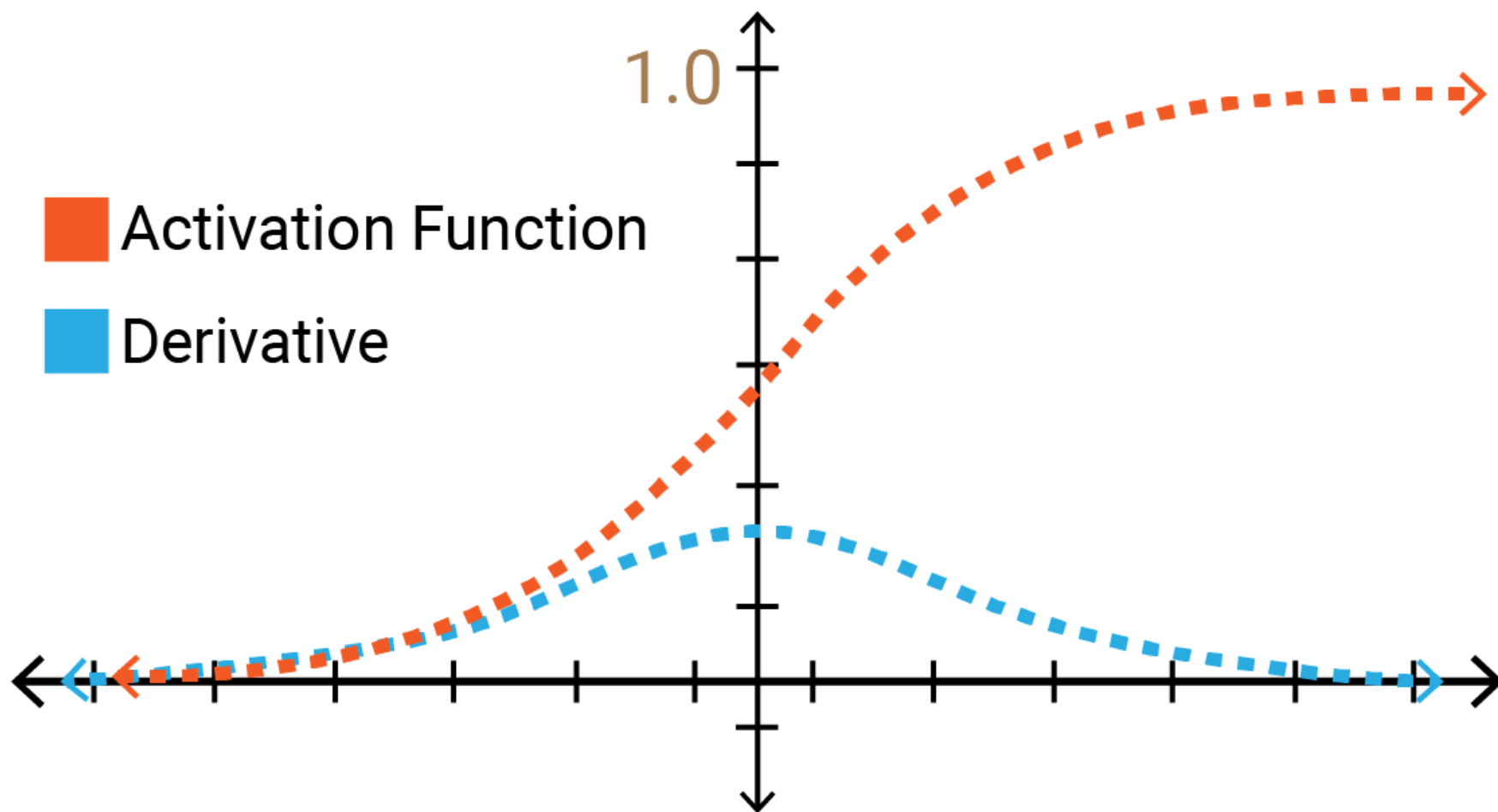
Key Motivation: Assume that only a half of the neurons are activated in each layer.

He's Xavier initialization: $N(0, 1)/\sqrt{n_{in}/2}$.

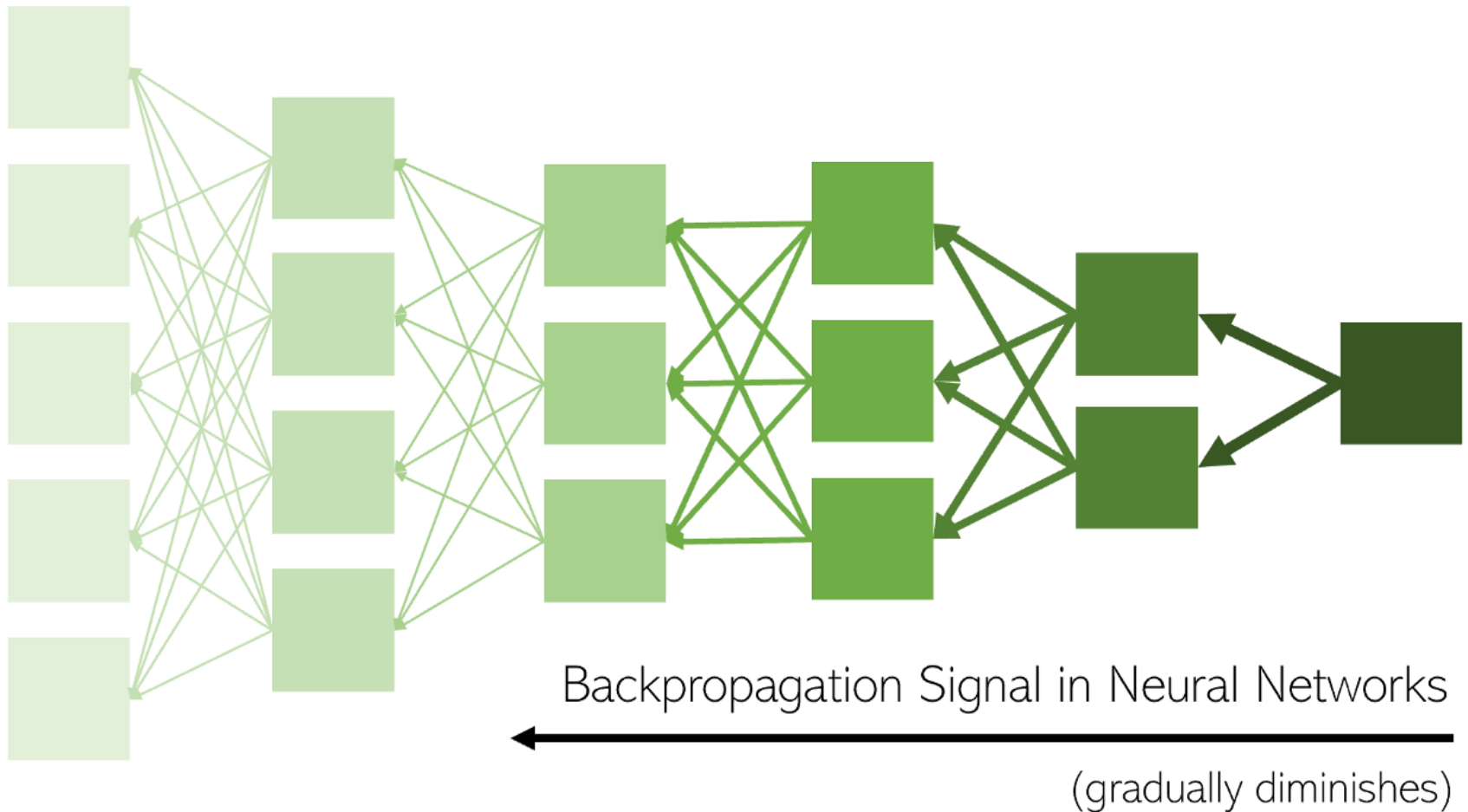


Bach Normalization

Motivation: Vanishing Gradients



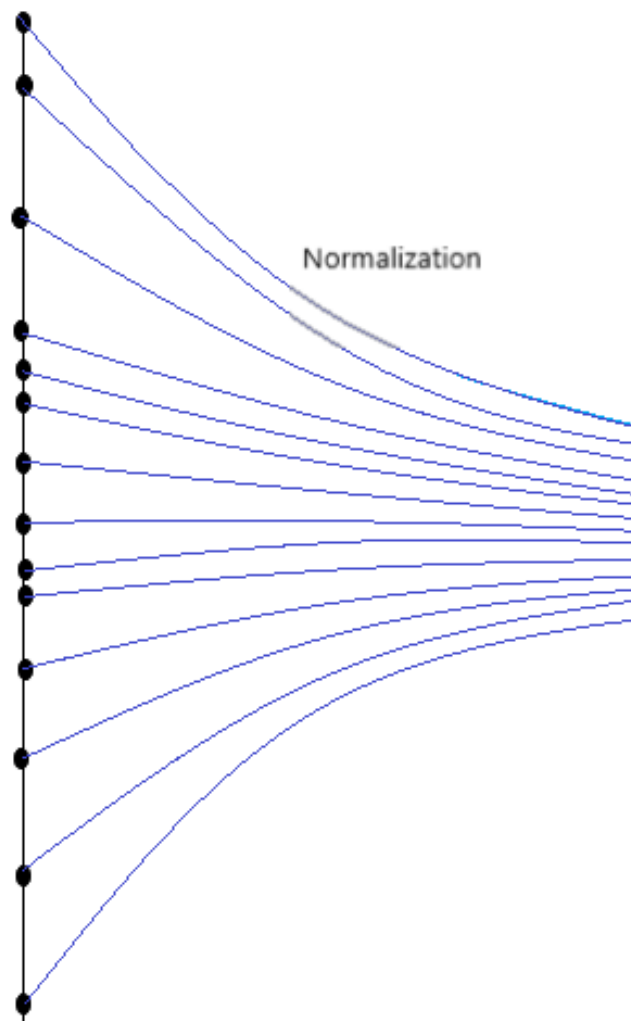
Motivation: Vanishing Gradients



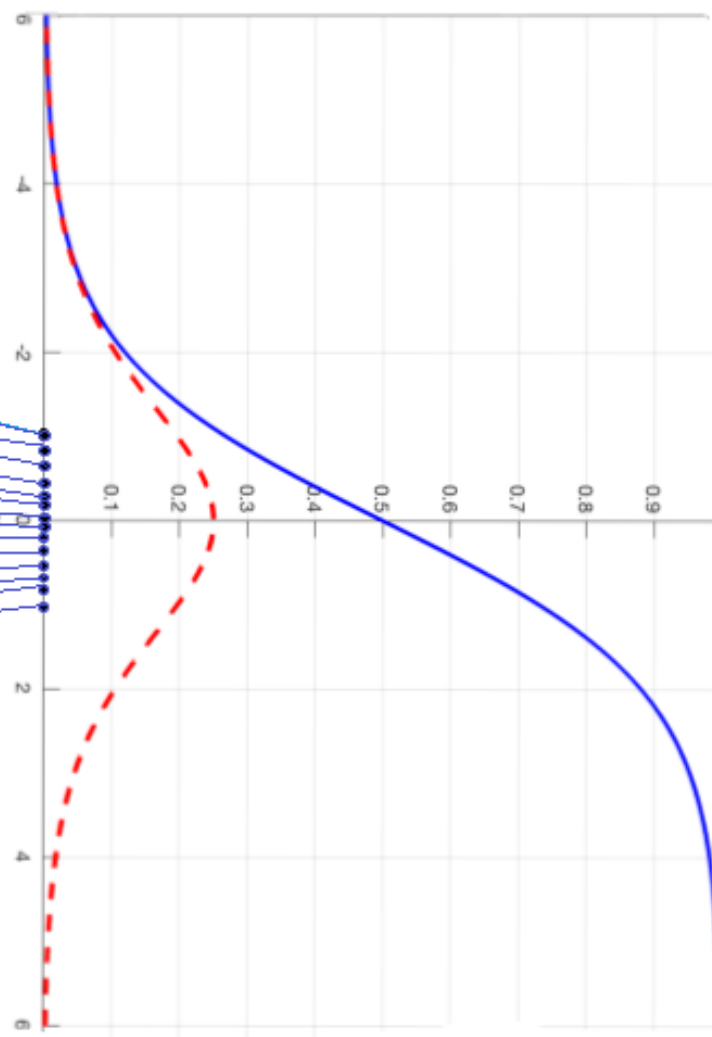
Motivation: Vanishing Gradients



Activation Inputs



Sigmoid Activation and Gradient



Consider an output of a linear layer:

$$y = Wx$$

The following scenarios are challenging:

1. $E[x] \neq 0$.
2. $Var[x]$ is large.

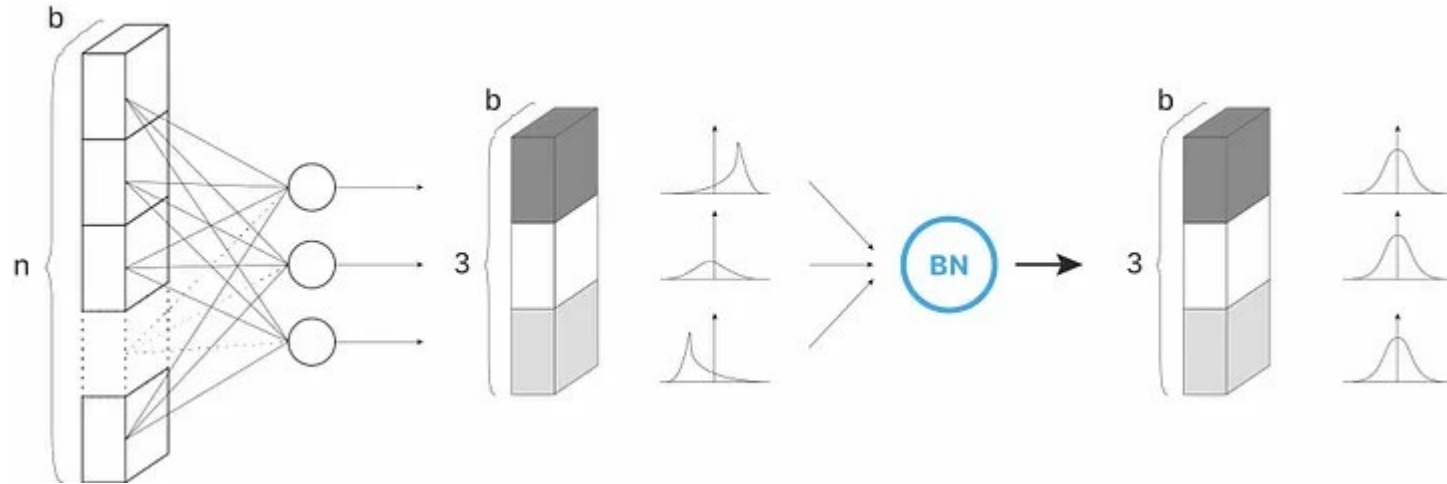
We WANT

1. $E[x] = 0$.
2. $Var[x] = 1$.

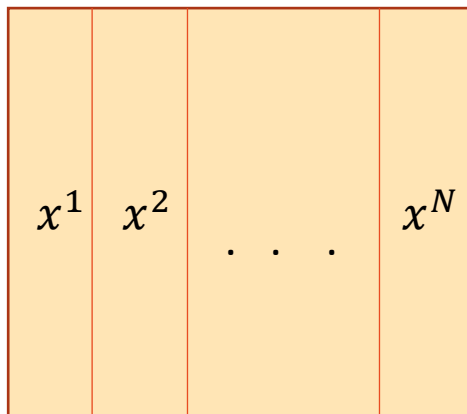
Batch Normalization is to normalize **each dimension** of the activations at some layer:

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{Var[x^{(k)}]}}.$$

Batch Normalization



Input data batch : $x^1, \dots, x^N \in \mathbb{R}^d$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_j^i$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_j^i - \mu_j)^2$$

$$\hat{x}_j^i = \frac{x_j^i - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Input data batch : $x^1, \dots, x^N \in R^d$, Learnable scale γ and β .

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_j^i$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_j^i - \mu_j)^2$$

$$\hat{x}_j^i = \frac{x_j^i - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

$$y_j^i = \gamma_j \hat{x}_j^i + \beta_j$$

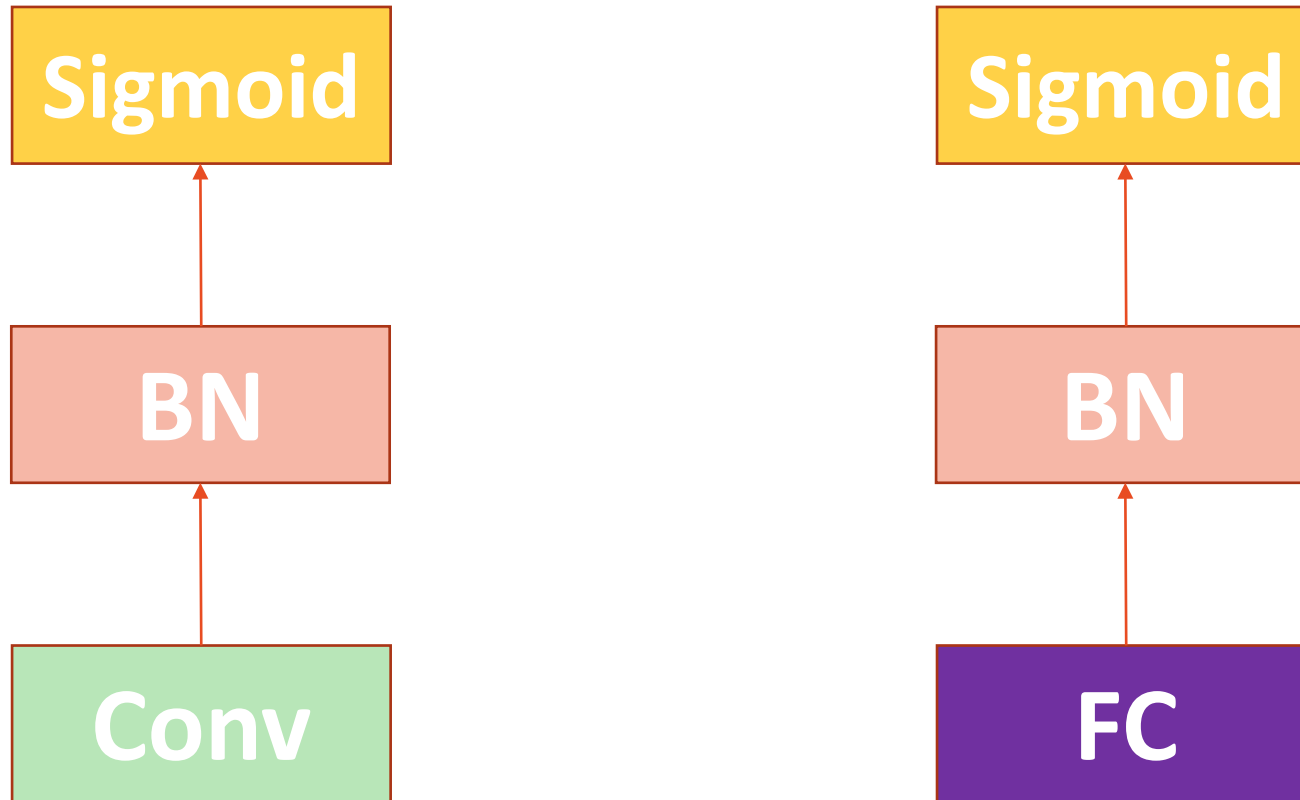
$$\mu_j = E[\mu_j^{train}]$$

$$\sigma_j^2 = E[(\sigma_j^2)^{train}]$$

$$\hat{x}_j^i = \frac{x_j^i - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

$$y_j^i = \gamma_j \hat{x}_j^i + \beta_j$$

It is a linear operator during testing.



Suggested Reading: [Batch normalization in 3 levels of understanding](#)

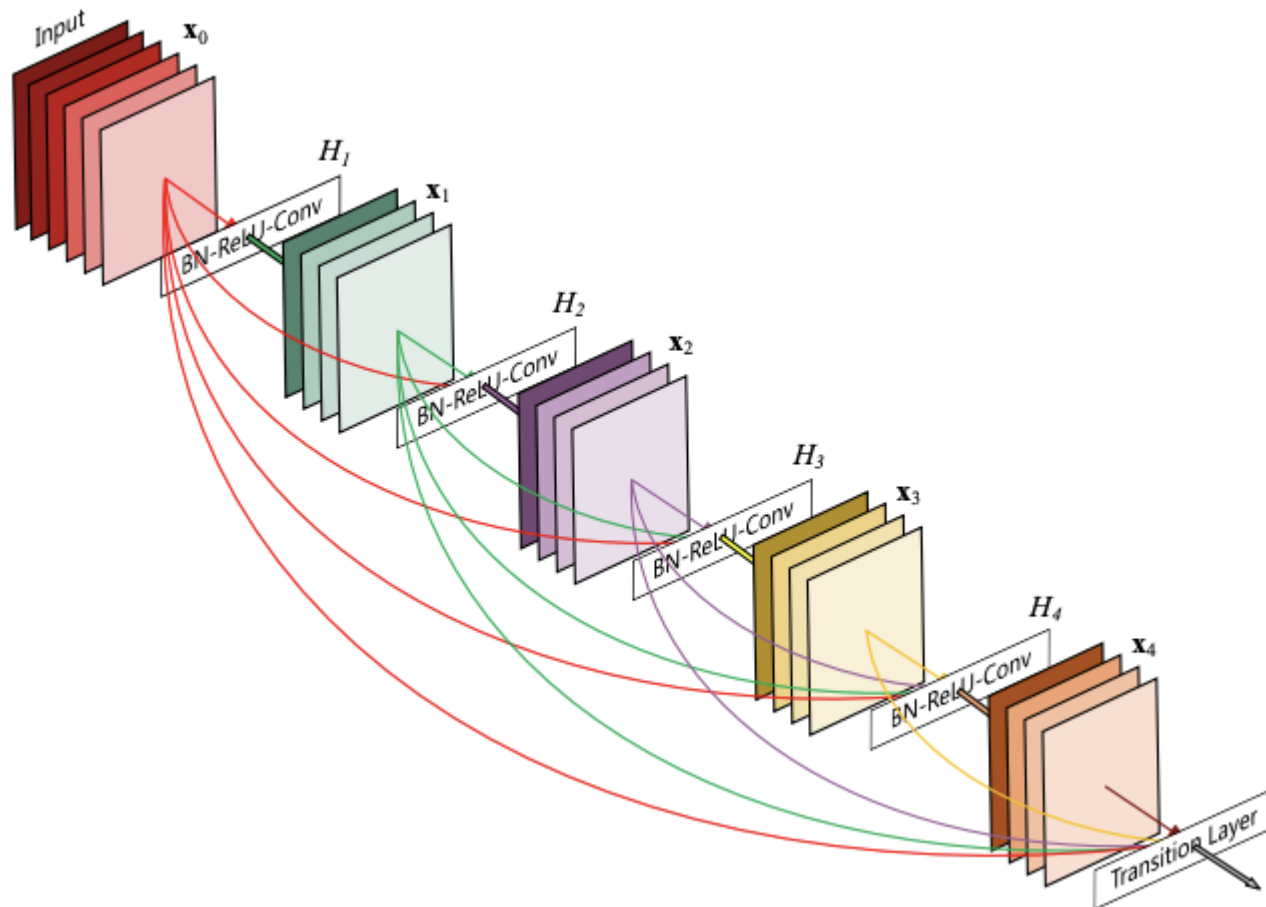
Why does Batch Normalization work?

<https://www.youtube.com/watch?v=nUUqwaxLnWs>

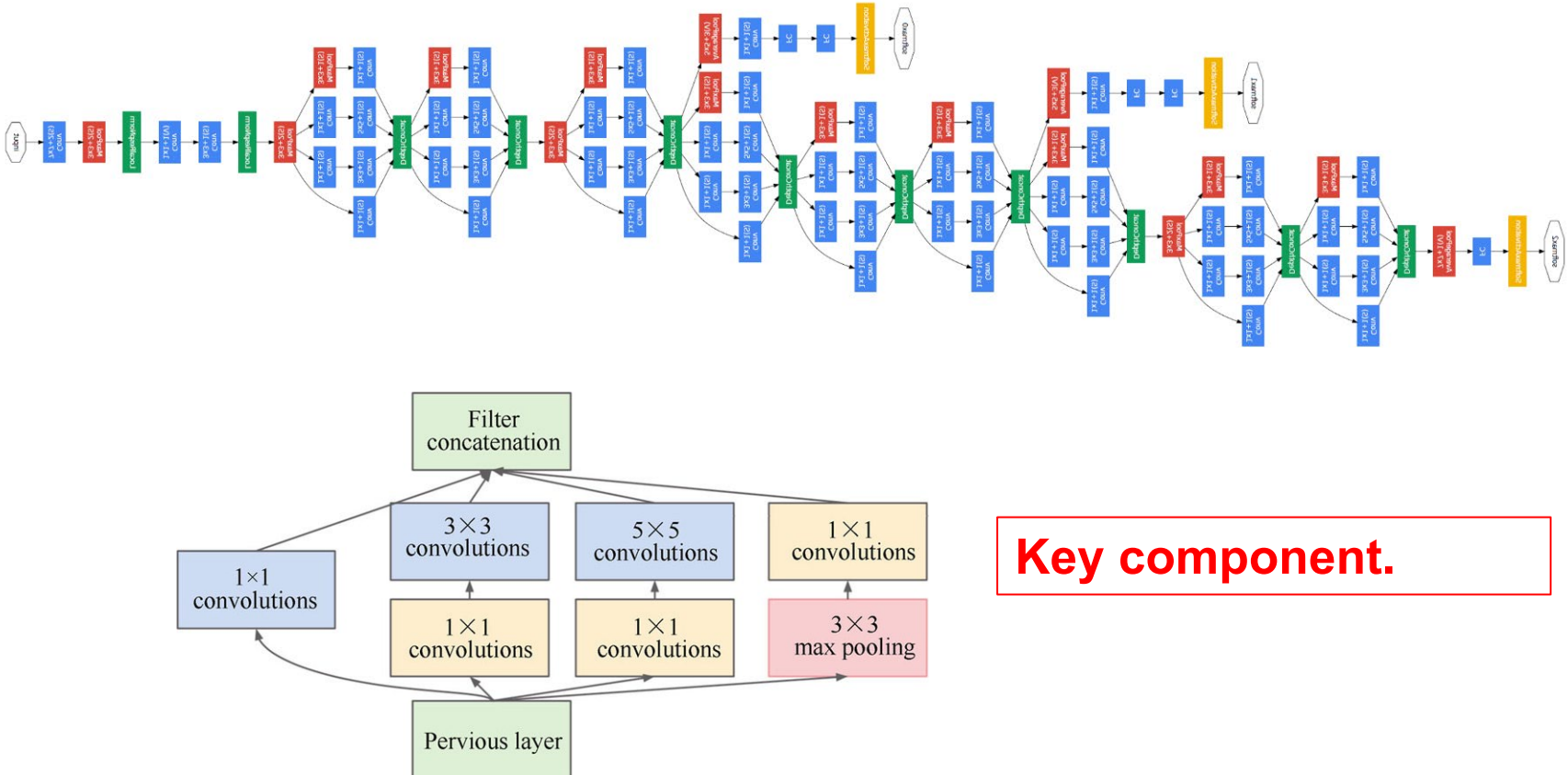
<https://www.youtube.com/watch?v=DtEq44FTPM4>

Other Famous CNN Architectures

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017).
Densely connected convolutional networks. CVPR



Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., ... & Rabinovich, A. (2015). Going deeper with convolutions. CVPR



Key component.